

FC7xxx ADC User Manual

Rev. 0.5

Revision History

Revision	Date	Changes
0.1	2023/07/14	Initial release
0.4	2023/11/17	Updated for MCAL V0.4.0 release
0.5	2024/01/17	Updated for MCAL V0.5.0 release

Table of Contents

Chapter 1 ADC Introduction.....	5
1.1 Requirement Tracing.....	5
1.2 Hardware Summary.....	5
Chapter 2 Software Design	6
2.1 Rejected Requirements	6
2.2 File Structure.....	12
2.3 Macros	12
2.3.1 Macros in Adc.h	12
2.3.2 Macros in Adc_types.h	15
2.3.3 Macros in Adc_version.h.....	15
2.3.4 Macros in Adc_Cfg.h.....	16
2.3.5 Macros in Adc_CfgDefines.h	16
2.4 Enums	20
2.4.1 Enumerations in Adc_Types.h	20
2.4.2 Enumerations in Adc_Hw_Types.h.....	23
2.4.3 Enumerations in Adc_Ptimer_Hw_Types.h	25
2.5 Typedefs	26
2.5.1 Typedefs in Adc_Types.h.....	26
2.6 Structures	28
2.6.1 Adc_ConfigType	28
2.6.2 Adc_HwUnitConfigurationType	29
2.6.3 Adc_GroupConfigurationType	29
2.6.4 Adc_ChannelConfigurationType.....	30
2.6.5 Adc_UnitStatusType	31
2.6.6 Adc_GroupStatusType.....	31
2.7 API Functions	32
2.7.1 Functions in Adc.h.....	32
2.8 Hardware Functions	35
2.8.1 Functions in Adc_Hw.h	35
2.8.2 Functions in Adc_Lld.h	37
2.8.3 Functions in Adc_Ptimer_Hw.h.....	41
2.9 API Sequence Diagram	42
2.9.1 Initialization of the ADC Driver	42
2.9.2 De-Initialization of the ADC Driver	43
2.9.3 Software Triggered One-Shot Conversion without Notification	44
2.9.4 Software Triggered Continuous Conversion with Notification.....	45

2.9.5	Hardware Triggered One-Shot Conversion with Notification	46
2.9.6	HW Trigger - One-Shot Conversion - Linear Streaming	47
2.9.7	No Priority Mechanism – No Queuing	48
2.9.8	No Priority Mechanism – SW Queuing	49
2.9.9	HW_SW Priority Mechanism – SW Queuing	50
Chapter 3 Tresos Configuration Items		51
3.1	Container Inclusion Relation	51
3.1.1	IMPLEMENTATION_CONFIG_VARIANT	51
3.1.2	AdcConfigSet	52
3.1.3	AdcGeneral	53
3.1.4	AdcInterrupt	53
3.1.5	AdcPublishedInformation	54
3.1.6	AutosarExt	54
3.1.7	AdcDemEventParameterRefs	54
3.2	Containers and Variables	55
3.2.1	IMPLEMENTATION_CONFIG_VARIANT	55
3.2.2	AdcConfigSet	55
3.2.3	AdcGeneral	72
3.2.4	AdcPublishedInformation	79
3.2.5	CommonPublishedInformation	80
3.2.6	AutosarExt	81
3.2.7	AdcDemEventParameterRefs	83
Chapter 4 Configuration Guides		84
4.1	Configuration Item Constraint	84
4.2	ADC Usage Common Steps	84

Chapter 1 ADC Introduction

1.1 Requirement Tracing

The design of this module follows the specifications of the ADC driver specified in AUTOSAR Classic Platform Release R20-11. For detailed requirements, refer to the *AUTOSAR_SWS_ADCDriver*.

1.2 Hardware Summary

The ADC driver is responsible to initialize and control the Analog to Digital module. The ADC driver provides user interfaces to configure the ADC parameters specified by *AUTOSAR_SWS_ADCDriver*. The ADC driver works on the ADC group. An ADC group is the combination of several ADC channel. And an ADC channel is the logical ID of an ADC hardware input which is assigned to an MCU pin. The ADC driver also includes configurations of channel delays, which is supported by the Ptimer module.

The FC7300 series has 4 ADC instances, and each instance has 32 logical ADC channels. There are 32 external physical channels and 5 internal physical channels for each instance which can be assigned to these logical channels.

Chapter 2 Software Design

2.1 Rejected Requirements

Rejected Requirement 1 **SWS_Adc_00341**

Description	If the priority mechanism is supported by the hardware: The ADC module shall support the static configuration option ADC_PRIORITY_HW to enable the priority mechanism using only the hardware priority mechanism.
Rejection Reason	Our ADC module does not support hardware priority mechanism.

Rejected Requirement 2 **SWS_Adc_00339**

Description	If hardware priority mechanism is supported and selected: The ADC module shall allow the mapping of the configured priority levels (0-255) to the available hardware priority levels.
Rejection Reason	Our ADC module does not support hardware priority mechanism.

Rejected Requirement 3 **SWS_Adc_00337**

Description	The hardware prioritization mechanism shall be used in case of hardware triggered conversion requests.
Rejection Reason	Our ADC module does not support hardware priority mechanism.

Rejected Requirement 3 **SWS_Adc_00462**

Description	The ADCDriver shall support power state changes and its APIs when the corresponding configuration parameter AdcLowPowerStatesSupport is set to TRUE.
Rejection Reason	Our ADC module does not support power state change.

Rejected Requirement 3 **SWS_Adc_00463**

Description	If the parameter AdcLowPowerStatesSupport is enabled then the APIs Adc_PreparePowerState, Adc_SetPowerState, Adc_GetCurrentPowerState, Adc_GetTargetPowerState shall be generated and shall be used to manage and get informations on power state transitions.
Rejection Reason	Our ADC module does not support power state change.

Rejected Requirement 3 **SWS_Adc_00464**

Description	The APIs Adc_GetTargetPowerState and Adc_GetCurrentPowerState shall be respectively used to gather information on the requested and the target ADC power states.
Rejection Reason	Our ADC module does not support power state change.

Rejected Requirement 3 **SWS_Adc_00465**

Description	The API Adc_PreparePowerState shall be used to start a power state transition.
Rejection Reason	Our ADC module does not support power state change.

Rejected Requirement 3 SWS_Adc_00466

Description	After preparation for a power state is achieved by API Adc_PreparePowerState then the API Adc_SetPowerState shall be used to achieve the requested power state of the ADC module.
Rejection Reason	Our ADC module does not support power state change.

Rejected Requirement 3 SWS_Adc_00467

Description	ADC Driver shall keep track of the call order of the APIs Adc_SetPowerState and Adc_PreparePowerState. In case the first one is called before the second one is called, a DET entry shall be stored and the action shall not be executed.
Rejection Reason	Our ADC module does not support power state change.

Rejected Requirement 3 SWS_Adc_00469

Description	The Adc Module shall keep track of the current and of the target powerstate if the parameter AdcLowPowerStatesSupport is set to TRUE.
Rejection Reason	Our ADC module does not support power state change.

Rejected Requirement 3 SWS_Adc_00470

Description	After the Initialization the power state of the module shall be always FULL POWER if the AdcLowPowerStatesSupport is set to TRUE.
Rejection Reason	Our ADC module does not support power state change.

Rejected Requirement 3 SWS_Adc_00471

Description	The ADC Driver shall support synchronuous and asynchronous power state transitions, depending on the value of the configuration parameter AdcPowerStateAsynchTransitionMode.
Rejection Reason	Our ADC module does not support power state change.

Rejected Requirement 3 SWS_Adc_00472

Description	In case the configuration parameter AdcPowerStateAsynchTransitionMode is set to FALSE, the preparation process and the setting process shall be considered concluded as soon as the respective APIs return.
Rejection Reason	Our ADC module does not support power state change.

Rejected Requirement 3 SWS_Adc_00473

Description	In case the configuration parameter AdcPowerStateAsynchTransitionMode is set to TRUE, the preparation process shall continue in background after the relative API returns and its completion shall be notified by means of the configured callback.
Rejection Reason	Our ADC module does not support power state change.

Rejected Requirement 3 SWS_Adc_00517

Description	Name: Adc_GroupDefType Type: implementation_specific Description: Type for assignment of channels to a channel group (this is not an API type)
Rejection Reason	In this implementation, the group assignment is implemented by an array of Adc_ChannelType.

Rejected Requirement 4 SWS_Adc_00527

Description	Name: Adc_PowerStateRequestResultType Type: Enumeration Range: ADC_SERVICE_ACCEPTED Power state change executed. ADC_NOT_INIT ADC Module not initialized. ADC_SEQUENCE_ERROR Wrong API call sequence. ADC_HW_FAILURE The HW module has a failure which prevents it to enter the required power state. ADC_POWER_STATE_NOT_SUPP ADC Module does not support the requested power state. ADC_TRANS_NOT_POSSIBLE ADC Module cannot transition directly from the current power state to the requested power state or the HW peripheral is still busy. Description: Result of the requests related to power state transitions.
Rejection Reason	Our ADC module does not support power state change.

Rejected Requirement 3 SWS_Adc_91000

Description	Service name: Adc_SetupResultBuffer Syntax: Std_ReturnType Adc_SetupResultBuffer(Adc_GroupType Group, Adc_ValueGroupType* DataBufferPtr) Service ID[hex]: 0x0c Sync/Async: Asynchronous Reentrancy: Reentrant Parameters (in): Group Numeric ID of requested ADC channel group. DataBufferPtr pointer to result data buffer Parameters (inout): None Parameters (out): None Return value: Std_ReturnType E_OK : result buffer pointer initialized correctly E_NOT_OK: operation failed or development error occurred Description: Initializes ADC driver with the group specific result buffer start address where the conversion results will be stored. The application has to ensure that the application buffer, where DataBufferPtr points to, can hold all the conversion results of the specified
--------------------	---

	group. The initialization with <code>Adc_SetupResultBuffer</code> is required after reset, before a group conversion can be started.
Rejection Reason	This function is implemented synchronous instead of asynchronous because the <code>Adc</code> module is implemted as Type II instead of Type IV.

Rejected Requirement 3 SWS_Adc_00358	
Description	The ADC module' s environment shall not call the function <code>Adc_Delnit</code> while any group is not in state <code>ADC_IDLE</code> .
Rejection Reason	This SWS applies restrictions on SWC, not MCAL

Rejected Requirement 3 SWS_Adc_00146	
Description	The ADC module' s environment shall only call <code>Adc_StartGroupConversion</code> for groups configured with software trigger source.
Rejection Reason	This SWS applies restrictions on SWC, not MCAL

Rejected Requirement 3 SWS_Adc_00283	
Description	The ADC module' s environment shall only call the function <code>Adc_StopGroupConversion</code> for groups configured with trigger source software.
Rejection Reason	This SWS applies restrictions on SWC, not MCAL

Rejected Requirement 3 SWS_Adc_91001	
Description	Service name: <code>Adc_EnableHardwareTrigger</code> Syntax: <pre>void Adc_EnableHardwareTrigger(Adc_GroupType Group)</pre> Service ID[hex]: 0x05 Sync/Async: Asynchronous Reentrancy: Reentrant Parameters (in): Group Numeric ID of requested ADC Channel group. Parameters (inout): None Parameters (out): None Return value: None Description: Enables the hardware trigger for the requested ADC Channel group.
Rejection Reason	This function is implemented synchronous instead of asynchronous because the <code>Adc</code> module is implemted as Type II instead of Type IV.

Rejected Requirement 3 SWS_Adc_00273	
Description	The ADC module' s environment shall guarantee that no concurrent conversions take place on the same HW Unit (happening of different hardware triggers at the same time).
Rejection Reason	This SWS applies restrictions on SWC, not MCAL

Rejected Requirement 3 SWS_Adc_00120	
Description	The ADC module' s environment shall only call the function Adc_EnableHardwareTrigger for groups configured in hardware trigger mode (see AdcGroupTriggSrc).
Rejection Reason	This SWS applies restrictions on SWC, not MCAL

Rejected Requirement 3 SWS_Adc_91002	
Description	Service name: Adc_DisableHardwareTrigger Syntax: <pre>void Adc_DisableHardwareTrigger(Adc_GroupType Group)</pre> Service ID[hex]: 0x06 Sync/Async: Asynchronous Reentrancy: Reentrant Parameters (in): Group Numeric ID of requested ADC Channel group. Parameters (inout): None Parameters (out): None Return value: None Description: Disables the hardware trigger for the requested ADC Channel group.
Rejection Reason	This function is implemented synchronous instead of asynchronous because the Adc module is implemted as Type II instead of Type IV.

Rejected Requirement 3 SWS_Adc_00121	
Description	The ADC module' s environment shall only call the function Adc_DisableHardwareTrigger for groups configured in hardware trigger mode (see AdcGroupTriggSrc).
Rejection Reason	This SWS applies restrictions on SWC, not MCAL

Rejected Requirement 3 SWS_Adc_91003	
Description	Service name: Adc_EnableGroupNotification Syntax: <pre>void Adc_EnableGroupNotification(Adc_GroupType Group)</pre> Service ID[hex]: 0x07 Sync/Async: Asynchronous Reentrancy: Reentrant Parameters (in): Group Numeric ID of requested ADC Channel group. Parameters (inout): None Parameters (out): None Return value: None Description: Enables the notification mechanism for the requested ADC Channel group.

Rejection Reason	This function is implemented synchronous instead of asynchronous because the Adc module is implemted as Type II instead of Type IV.
-------------------------	---

Rejected Requirement 3 SWS_Adc_91004	
Description	Service name: Adc_DisableGroupNotification Syntax: <pre>void Adc_DisableGroupNotification(Adc_GroupType Group)</pre> Service ID[hex]: 0x08 Sync/Async: Asynchronous Reentrancy: Reentrant Parameters (in): Group Numeric ID of requested ADC Channel group. Parameters (inout): None Parameters (out): None Return value: None Description: Disables the notification mechanism for the requested ADC Channel group.
Rejection Reason	This function is implemented synchronous instead of asynchronous because the Adc module is implemted as Type II instead of Type IV.

Rejected Requirement 3 SWS_Adc_00305	
Description	To guarantee consistent returned values, it is assumed that ADC group conversion is always started (or enabled in case of HW group) successfully by SW before status polling begins.
Rejection Reason	This SWS applies restrictions on SWC, not MCAL

Rejected Requirement 3 SWS_Adc_00219	
Description	The ADC module' s environment shall guarantee the consistency of the data that has been read by checking the return value of Adc_GetGroupStatus.
Rejection Reason	This SWS applies restrictions on SWC, not MCAL

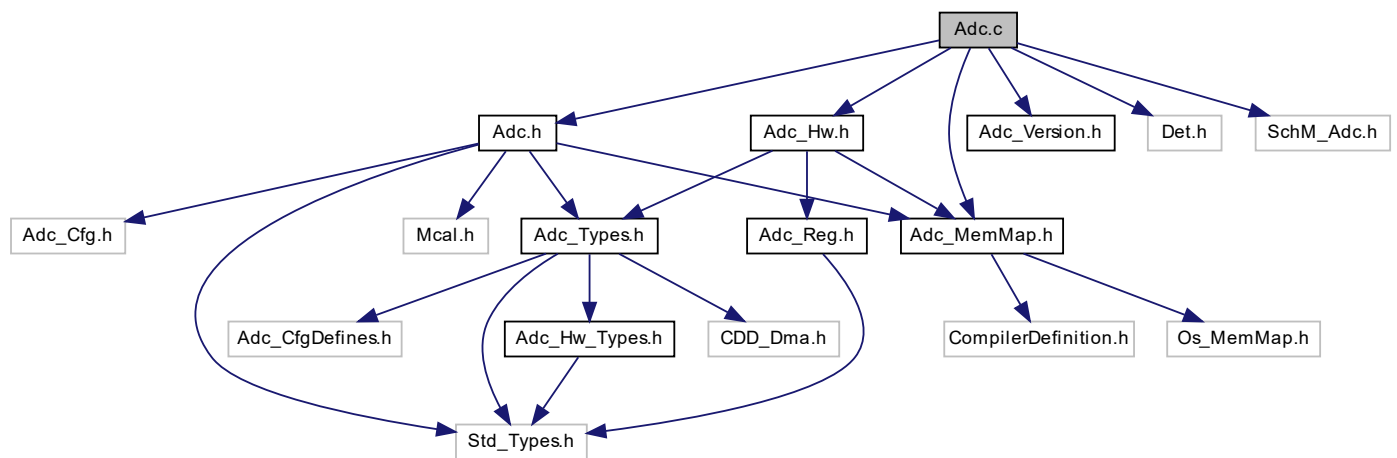
Rejected Requirement 3 SWS_Adc_CONSTR_00001	
Description	The ECUC partitions referenced by AdcKernelEcucPartitionRef shall be a subset of the ECUC partitions referenced by AdcEcucPartitionRef.
Rejection Reason	The Adc module is implemented as Type II and thus AdcKernelEcucPartitionRef cannot be configured.

Rejected Requirement 3 SWS_Adc_CONSTR_00003	
Description	If AdcEcucPartitionRef references one or more ECUC partitions, AdcKernelEcucPartitionRef shall have a multiplicity of one and reference one of these ECUC partitions as well.
Rejection Reason	The Adc module is implemented as Type II and thus AdcKernelEcucPartitionRef cannot be configured.

Rejected Requirement 3 SWS_Adc_CONSTR_00002	
Description	The ECUC partitions referenced by AdcGroupEcucPartitionRef shall be a subset of the ECUC partitions referenced by AdcEcucPartitionRef.
Rejection Reason	The Adc module is implemented as Type II and thus AdcGroupEcucPartitionRef cannot be configured.

Rejected Requirement 3 SWS_Adc_CONSTR_00004	
Description	If AdcEcucPartitionRef references one or more ECUC partitions, AdcGroupEcucPartitionRef shall have a multiplicity of greater than zero and reference one or several of these ECUC partitions as well.
Rejection Reason	The Adc module is implemented as Type II and thus AdcGroupEcucPartitionRef cannot be configured.

2.2 File Structure



2.3 Macros

2.3.1 Macros in Adc.h

- **#define ADC_E_UNINIT** ((uint8)0x0AU)
API service used without ADC module initialization.
- **#define ADC_E_BUSY** ((uint8)0x0BU)
ADC module is busy with a running operation.
- **#define ADC_E_IDLE** ((uint8)0x0CU)
ADC module is in idle state.
- **#define ADC_E_ALREADY_INITIALIZED** ((uint8)0x0DU)
The ADC module is already initialized.

- **#define ADC_E_PARAM_CONFIG ((uint8)0x0EU)**
The ADC module is not properly configured.
- **#define ADC_E_PARAM_POINTER ((uint8)0x14U)**
API service is called using an invalid pointer (e.g. the pointer should not be NULL).
- **#define ADC_E_PARAM_GROUP ((uint8)0x15U)**
API service used with an invalid ADC group.
- **#define ADC_E_WRONG_CONV_MODE ((uint8)0x16U)**
API service used with an invalid ADC Conversion Mode.
- **#define ADC_E_WRONG_TRIGG_SRC ((uint8)0x17U)**
API service used with an invalid ADC Trigger Source.
- **#define ADC_E_NOTIF_CAPABILITY ((uint8)0x18U)**
Check the notification capability of a group.
- **#define ADC_E_BUFFER_UNINIT ((uint8)0x19U)**
API service used without initializing the buffer.
- **#define ADC_E_NOT_DISENGAGED ((uint8)0x1AU)**
One or more ADC group/channel not in IDLE state.
- **#define ADC_E_POWER_STATE_NOT_SUPPORTED ((uint8)0x1BU)**
Unsupported power state request.
- **#define ADC_E_TRANSITION_NOT_POSSIBLE ((uint8)0x1CU)**
Requested power state cannot be reached directly.
- **#define ADC_E_PERIPHERAL_NOT_PREPARED ((uint8)0x1DU)**
ADC not prepared for target power state.
- **#define ADC_E_QUEUE_FULL ((uint8)0x20U)**
The `Adc_StartGroupConversion` and `Adc_EnableHardwareTrigger` services cannot queue another conversion (queue is full) .
- **#define ADC_E_PARAM_UNIT ((uint8)0x27U)**
API service called using a wrong ADC unit.

- **#define ADC_INIT_ID 0x00U**
API service ID for Adc_Init function.
- **#define ADC_DEINIT_ID 0x01U**
API service ID for Adc_DeInit function.
- **#define ADC_STARTGROUPCONVERSION_ID 0x02U**
API service ID for Adc_StartGroupConversion function.
- **#define ADC_STOPGROUPCONVERSION_ID 0x03U**
API service ID for Adc_StopGroupConversion function.
- **#define ADC_VALUEREADGROUP_ID 0x04U**
API service ID for Adc_ReadGroup function.
- **#define ADC_ENABLEHARDWARETRIGGER_ID 0x05U**
API service ID for Adc_EnableHardwareTrigger function.
- **#define ADC_DISABLEHARDWARETRIGGER_ID 0x06U**
API service ID for Adc_DisableHardwareTrigger function.
- **#define ADC_ENABLEGROUPNOTIFICATION_ID 0x07U**
API service ID for Adc_EnableGroupNotification function.
- **#define ADC_DISABLEGROUPNOTIFICATION_ID 0x08U**
API service ID for Adc_DisableGroupNotification function.
- **#define ADC_GETGROUPSTATUS_ID 0x09U**
API service ID for Adc_GetGroupStatus function.
- **#define ADC_GETVERSIONINFO_ID 0x0AU**
API service ID for Adc_GetVersionInfo function.
- **#define ADC_GETSTREAMLASTPOINTER_ID 0x0BU**
API service ID for Adc_GetStreamLastPointer function.
- **#define ADC_SETUPRESULTBUFFER_ID 0x0CU**
API service ID for Adc_SetupResultBuffer function.

- **#define `ADC_SETPOWERSTATE_ID` 0x10U**
API service ID for `Adc_SetPowerState` function.
- **#define `ADC_GETCURRENTPOWERSTATE_ID` 0x11U**
API service ID for `Adc_GetCurrentPowerState` function.
- **#define `ADC_GETTARGETPOWERSTATE_ID` 0x12U**
API service ID for `Adc_GetTargetPowerState` function.
- **#define `ADC_PREPAREPOWERSTATE_ID` 0x13U**
API service ID for `Adc_PreparePowerState` function.

2.3.2 Macros in `Adc_types.h`

- **#define `ADC_BITS_RESOLUTION_8` 2U**
- **#define `ADC_BITS_RESOLUTION_10` 1U**
- **#define `ADC_BITS_RESOLUTION_12` 0U**
ADC Resolutions.
- **#define `ADC_CLOCK_DIVIDE_BY_1` 0U**
- **#define `ADC_CLOCK_DIVIDE_BY_2` 1U**
- **#define `ADC_CLOCK_DIVIDE_BY_4` 2U**
- **#define `ADC_CLOCK_DIVIDE_BY_8` 3U**
ADC Prescaler types.

2.3.3 Macros in `Adc_version.h`

- **#define `ADC_AR_RELEASE_MAJOR_VERSION` 4**
- **#define `ADC_AR_RELEASE_MINOR_VERSION` 6**
- **#define `ADC_AR_RELEASE_REVISION_VERSION` 0**
- **#define `ADC_SW_MAJOR_VERSION` 0**

- `#define ADC_SW_MINOR_VERSION 5`
- `#define ADC_SW_PATCH_VERSION 0`
- `#define ADC_VENDOR_ID 174`
- `#define ADC_MODULE_ID 123`

2.3.4 Macros in `Adc_Cfg.h`

- `#define ADC_CFG_AR_RELEASE_MAJOR_VERSION 4`
- `#define ADC_CFG_AR_RELEASE_MINOR_VERSION 6`
- `#define ADC_CFG_AR_RELEASE_REVISION_VERSION 0`
- `#define ADC_CFG_SW_MAJOR_VERSION 0`
- `#define ADC_CFG_SW_MINOR_VERSION 5`
- `#define ADC_CFG_SW_PATCH_VERSION 0`
- `#define ADC_CFG_VENDOR_ID 174`
- `#define ADC_CFGSET_GROUP_x_CHANNELS n`
Number of channels configured for each group.

2.3.5 Macros in `Adc_CfgDefines.h`

- `#define ADC_MULTICORE_SUPPORT (STD_ON)/(STD_OFF)`
Enables or disables Multicore feature in the ADC driver ((STD_ON)/(STD_OFF)).
- `#define ADC_UNIT_0 0U`
ADC unit 0 available.
- `#define ADC_UNIT_1 1U`
ADC unit 1 available.
- `#define ADC_UNIT_2 2U`
ADC unit 2 available.

- **#define ADC_UNIT_3 3U**
ADC unit 3 available.
- **#define ADC_MAX_PARTITIONS 3U**
Maximum number of configured partitions.
- **#define ADC_NUM_SC_REGISTER 32U**
Number of SC1n registers available.
- **#define ADC_NUM_SC_REGISTER_USED 32U**
Number of SC1n registers used.
- **#define ADC_PTIMER_NUM_DELAY_REGISTER 8U**
Ptimer support 8 delay register.
- **#define ADC_HW_TRIGGER_API (STD_ON)/(STD_OFF)**
Adds/removes ((STD_ON)/(STD_OFF)) the services `Adc_EnableHardwareTrigger()` and `Adc_DisableHardwareTrigger()` from the code.
- **#define ADC_ENABLE_LIMIT_CHECK (STD_ON)/(STD_OFF)**
Enables or disables limit checking feature in the ADC driver.
- **#define ADC_GRP_NOTIF_CAPABILITY (STD_ON)/(STD_OFF)**
Group notification mechanism is/isn't ((STD_ON)/(STD_OFF)) available at runtime.
- **#define ADC_PRIORITY_NONE 0U**
- **#define ADC_PRIORITY_HW 1U**
- **#define ADC_PRIORITY_HW_SW 2U**
Type of prioritization mechanism `ADC_PRIORITY_HW`, `ADC_PRIORITY_HW_SW` and `ADC_PRIORITY_NONE`.
- **#define ADC_PRIORITY_IMPLEMENTATION (ADC_PRIORITY_NONE)/(ADC_PRIORITY_HW_SW)**
Priority implementation.
- **#define ADC_ALIGN_RIGHT 0U**
- **#define ADC_ALIGN_LEFT 1U**
Type of alignment mechanism `ADC_ALIGN_RIGHT`, `ADC_ALIGN_LEFT`.

- **#define `ADC_RESULT_ALIGNMENT` (`ADC_ALIGN_RIGHT`)**
Result Alignment.
- **#define `ADC_RESULT_U8` `0U`**
- **#define `ADC_RESULT_U16` `1U`**
- **#define `ADC_RESULT_U32` `2U`**
Type of Adc result bit width `ADC_RESULT_U8`, `ADC_RESULT_U16`, `ADC_RESULT_U32`.
- **#define `ADC_RESULT_TYPE` (`ADC_RESULT_U16`)**
Result bit width.
- **#define `ADC_TIMEOUT_COUNTER` (`0x0000ffffUL`)**
This is a timeout value which is used to wait till: the conversion is not aborted; ADC hardware is not entered in power down state; ADC hardware is not entered in idle state.
- **#define `ADC_DMA_TRANSFER_TIMEOUT` (`0x0000ffffUL`)**
This is a timeout value which is used to wait till DMA transferring finishes.
- **#define `ADC_DMA_SUPPORT` (`STD_OFF`)**
Add/Remove feature to support DMA transfer.
- **#define `ADC_MAX_HW_UNITS` `4U`**
Max number of ADCDig HW units.
- **#define `ADC_PTIMER_CHANNELS_U8` `4U`**
Define the number of maximum channels.
- **#define `ADC_PTIMER_NUM_MODULES` `2U`**
Number Ptimer module.
- **#define `ADC_DISABLE_DEM_REPORT_ERROR_STATUS` (`STD_ON`)**
Switch to enable/disable the production error reporting.
- **#define `ADC_UNIT_0_ISR_USED`**
#define `ADC_UNIT_1_ISR_USED`
#define `ADC_UNIT_2_ISR_USED`

#define **ADC_UNIT_3_ISR_USED**

IRQ definition.

- #define **ADC_QUEUE_MAX_DEPTH_MAX** 3U
Max queue depth configured across all configset.
- #define **ADC_MAX_GROUPS** 1U
Max number of groups configured across all configset.
- #define **ADC_HW_QUEUE** (1U)
Max queue depth for hardware triggered conversion.
- #define **ADC_DEV_ERROR_DETECT** (STD_ON)
Development error detection enabled/disabled ((STD_ON)/(STD_OFF)).
- #define **ADC_VERSION_INFO_API** (STD_ON)/(STD_OFF)
Add/remove ((STD_ON)/(STD_OFF)) the service `Adc_GetVersionInfo()` from the code.
- #define **ADC_DEINIT_API** (STD_ON)/(STD_OFF)
Add/remove ((STD_ON)/(STD_OFF)) the service `Adc_DeInit()` from the code.
- #define **ADC_ENABLE_START_STOP_GROUP_API** (STD_ON)/(STD_OFF)
Add/remove ((STD_ON)/(STD_OFF)) the services `Adc_StartGroupConversion()` and `Adc_StopGroupConversion()` from the code.
- #define **ADC_READ_GROUP_API** (STD_ON)/(STD_OFF)
Add/Remove the services `Adc_ReadGroup()` from the code.
- #define **ADC_CONTINUOUS_NO_INTERRUPT_SUPPORT** (STD_ON)/(STD_OFF)
This switch is used to enable the AutosarExt ADC performance optimization feature.
- #define **ADC_PRESCALER_CONF_FEATURE** (STD_ON)/(STD_OFF)
Add/Remove feature to configure `Adc` clock prescaler.
- #define **ADC_SET_ADC_HARDWARE_AVERAGE_ONCE** (STD_ON)/(STD_OFF)
Add/Remove One time setting of Conversion time registers from `Init()` function.
- #define **ADC_ENABLE_QUEUEING** (STD_ON)/(STD_OFF)
This switch is used to enable the queue.

- #define **AdcHwUnit_0** (0)
Symbolic names of ADC Hardware units.
- #define **ADC_ID_CHANNEL_U8(Id)** (Id)
ADC channel ID.
- #define **ADC_QUEUE_INDEX_TYPE_MAXIMUM** 65535U
Maximum boundary for the Adc_QueueIndexType.
- #define **AdcChannel_1** 0
- #define **AdcChannel_0** 1
AUTOSAR Symbolic names of channels on all HW units.
- #define **AdcGroup_0** 0
Symbolic names of groups.
- #define **AdcConf_AdcGroup_AdcGroup_0** 0
Symbolic names of groups - TPS_ECUC_02108 compliant.
- #define **ADC_ENABLE_USER_MODE_SUPPORT** (STD_OFF)
Support for User mode. If this parameter has been configured to 'STD_ON', the ADC driver code can be executed from both supervisor and user mode.

2.4 Enums

2.4.1 Enumerations in Adc_Types.h

2.4.1.1 Adc_GroupConversionStateType

Enumeration	Adc_GroupConversionStateType	
Description	ADC group already converted type. Used to differentiate if group is already converted or not.	
Values	Value	Description
	ADC_NOT_YET_CONVERTED = 0U	Group not yet converted.
	ADC_ALREADY_CONVERTED = 1U	Group is already converted.

2.4.1.2 Adc_GroupAccessModeType

Enumeration	Adc_GroupAccessModeType	
Description	ADC group access Mode. Used for value received by Tresos interface configuration.	
Values	Value	Description
	ADC_ACCESS_MODE_SINGLE = 0U	Single access mode.
	ADC_ACCESS_MODE_STREAMING =	Streaming access mode

	1U	
--	----	--

2.4.1.3 Adc_GroupConvModeType

Enumeration	Adc_GroupConvModeType	
Description	ADC Group conversion mode. Used for value received by Tresos interface configuration.	
Values	Value	Description
	ADC_CONV_MODE_ONESHOT = 0U	One shot conversion mode.
	ADC_CONV_MODE_CONTINUOUS = 1U	Continuous conversion mode.

2.4.1.4 Adc_GroupReplacementType

Enumeration	Adc_GroupReplacementType	
Description	ADC group replacement. Used for value received by Tresos interface configuration.	
Values	Value	Description
	ADC_GROUP_REPL_ABORT_RESTART = 0U	Abort and restart of group.
	ADC_GROUP_REPL_SUSPEND_RESUME = 1U	Suspend and resuming of group.

2.4.1.5 Adc_StreamBufferModeType

Enumeration	Adc_StreamBufferModeType	
Description	ADC group streaming buffer mode. Used for value received by Tresos interface configuration.	
Values	Value	Description
	ADC_STREAM_BUFFER_LINEAR = 0U	Linear streaming.
	ADC_STREAM_BUFFER_CIRCULAR = 1U	Circular streaming.

2.4.1.6 Adc_TransferType

Enumeration	Adc_TransferType	
Description	Adc group data transfer type. Used for value received by Tresos interface configuration.	
Values	Value	Description
	ADC_TRANSFER_TYPE_INTERRUPT = 0U	Transfer by interrupt.
	ADC_TRANSFER_TYPE_DMA = 1U	Transfer by DMA.
	ADC_TRANSFER_TYPE_POLLING = 2U	Transfer by polling.

2.4.1.7 Adc_StatusType

Enumeration	Adc_StatusType	
Description	Indicates the ADC group status.	
Values	Value	Description
	ADC_IDLE = 0U	Group is in IDLE state.

	ADC_BUSY = 1U	Group is in BUSY state.
	ADC_COMPLETED = 2U	Group is in COMPLETED state.
	ADC_STREAM_COMPLETED = 3U	Group is in STREAM_COMPLETED state.

2.4.1.8 Adc_NotificationType

Enumeration	Adc_NotificationType	
Description	ADC group notification. Indicates if notification is enabled for the group.	
Values	Value	Description
	ADC_NOTIFICATION_DISABLED = 0U	Notification is disabled.
	ADC_NOTIFICATION_ENABLED = 1U	Notification is enabled.

2.4.1.9 Adc_HwTriggerSignalType

Enumeration	Adc_HwTriggerSignalType	
Description	ADC hardware trigger edge. Used for value received by Tresos interface configuration.	
Values	Value	Description
	ADC_HW_TRIG_RISING_EDGE = 0U	Rising edge.
	ADC_HW_TRIG_FALLING_EDGE = 1U	Falling edge.
	ADC_HW_TRIG_BOTH_EDGES = 2U	Falling and rising edge.

2.4.1.10 Adc_TriggerSourceType

Enumeration	Adc_TriggerSourceType	
Description	ADC hardware trigger source. Used for value received by Tresos interface configuration.	
Values	Value	Description
	ADC_TRIGG_SRC_SW = 0U	Rising edge.
	ADC_TRIGG_SRC_HW = 1U	Falling edge.

2.4.1.11 Adc_HwTriggeringType

Enumeration	Adc_HwTriggeringType	
Description	ADC Hardware trigger. Indicates if hardware trigger is enabled for group.	
Values	Value	Description
	ADC_HWTRIGGER_DISABLED = 0U	Hardware trigger is disabled.
	ADC_HWTRIGGER_ENABLED = 1U	Hardware trigger is enabled.

2.4.1.12 Adc_ChannelRangeSelectType

Enumeration	Adc_ChannelRangeSelectType	
Description	Range select values. Indicates which range select is used.	
Values	Value	Description
	ADC_RANGE_UNDER_LOW = 0U	Range below low limit - low limit value included
	ADC_RANGE_BETWEEN = 1U	Range between low limit and high limit - high limit value included.
	ADC_RANGE_OVER_HIGH = 2U	Range above high limit

	ADC_RANGE_ALWAYS = 3U	Complete range - independent from channel limit settings
	ADC_RANGE_NOT_UNDER_LOW = 4U	Range above low limit
	ADC_RANGE_NOT_BETWEEN = 5U	Range above high limit or below low limit - low limit value included
	ADC_RANGE_NOT_OVER_HIGH = 6U	Range below high limit - high limit value included

2.4.2 Enumerations in Adc_Hw_Types.h

2.4.2.1 Adc_AlignType

Enumeration	Adc_AlignType	
Description	Select the ADC result alignment.	
Values	Value	Description
	ADC_RESULT_ALIGN_RIGHT = 0U	ADC result is aligned in right.
	ADC_RESULT_ALIGN_LEFT = 1U	ADC result is aligned in left.

2.4.2.2 Adc_TrigModeType

Enumeration	Adc_TrigModeType	
Description	Select the ADC trigger mode.	
Values	Value	Description
	ADC_TRIGMODE_SW = 0U	ADC is triggered by software.
	ADC_TRIGMODE_RISING_EDGE = 1U	ADC is triggered by hardware trigger on rising edge.
	ADC_TRIGMODE_FALLING_EDGE = 2U	ADC is triggered by hardware trigger on falling edge.
	ADC_TRIGMODE_BOTH_EDGE = 3U	ADC is triggered by hardware trigger on both edges.
	ADC_TRIGMODE_HIGH_VOLTAGE = 4U	ADC is triggered when hardware trigger is high voltage.
	ADC_TRIGMODE_LOW_VOLTAGE = 5U	ADC is triggered when hardware trigger is low voltage.

2.4.2.3 Adc_SeqModeType

Enumeration	Adc_SeqModeType	
Description	Select the ADC sequence mode	
Values	Value	Description
	GPT_PREDEF_TIMER_1US_16BIT = 0x0U	1µs 16bit predef timer.
	GPT_PREDEF_TIMER_1US_24BIT	1µs 24bit predef timer.
	GPT_PREDEF_TIMER_1US_32BIT	1µs 32bit predef timer.
	GPT_PREDEF_TIMER_100US_32BIT	100µs 32bit predef timer.

2.4.2.4 Adc_SeqModeType

Enumeration	Adc_SeqModeType	
Description	Select the ADC sequence mode	
Values	Value	Description
	ADC_SEQMODE_SINGLE = 0U	ADC single sequence mode.
	ADC_SEQMODE_CONTINUOUS = 1U	ADC continuous mode.

	ADC_SEQMODE_DISCONTINUOUS_0 = 2U	ADC discontinuous 0 mode.
	ADC_SEQMODE_DISCONTINUOUS_1 = 3U	ADC discontinuous 1 mode.

2.4.2.5 Adc_OvrModeType

Enumeration	Adc_OvrModeType	
Description	Select the ADC overrun management mode	
Values	Value	Description
	ADC_OVERRUN_MODE_PRESERVE = 0U	Old data are preserved when ADC is overrun.
	ADC_OVERRUN_MODE_OVERWRITE = 1U	Old data are overwritten when ADC is overrun.

2.4.2.6 Adc_RefType

Enumeration	Adc_RefType	
Description	Select the ADC voltage reference source	
Values	Value	Description
	ADC_REF_INTERNAL = 0U	Select the internal voltage reference.
	ADC_REF_EXTERNAL = 1U	Select the external voltage reference.

2.4.2.7 Adc_AverageType

Enumeration	Adc_AverageType	
Description	Select the ADC hardware average samples.	
Values	Value	Description
	ADC_AVERAGE_4 = 0U	Result average by 4 samples.
	ADC_AVERAGE_8 = 1U	Result average by 8 samples.
	ADC_AVERAGE_16 = 2U	Result average by 16 samples.
	ADC_AVERAGE_32 = 3U	Result average by 32 samples.

2.4.2.8 Adc_TrigSrcType

Enumeration	Adc_TrigSrcType	
Description	The trigger source of the ADC instance.	
Values	Value	Description
	ADC_TRIGSRC_SMISC = 0U	Trigger source from PTIMER/TRGSEL with SMISC.
	ADC_TRIGSRC_TRGSEL = 2U	Trigger source from TRGSEL.

2.4.2.9 Adc_PreTriggerSelType

Enumeration	Adc_PreTriggerSelType	
Description	Select the ADC pretrigger source for discontinuous mode.	
Values	Value	Description

	ADC_PRETRIGGER_SEL_PTIMER = 0x00U	Ptimer pretrigger selected.
	ADC_PRETRIGGER_SEL_SW = 0x02U	Software pretrigger selected.

2.4.2.10 Adc_TriggerSelType

Enumeration	Adc_TriggerSelType	
Description	Select the ADC trigger source for discontinuous 1 mode.	
Values	Value	Description
	ADC_TRIGGER_SEL_PTIMER = 0x00U	Ptimer trigger selected.
	ADC_TRIGGER_SEL_TRGSEL = 0x01U	TRGSEL trigger selected.

2.4.2.11 Adc_SwPreTriggerType

Enumeration	Adc_SwPreTriggerType	
Description	Select the software pretrigger source.	
Values	Value	Description
	ADC_SW_PRETRIGGER_DISABLED = 0x00U	SW pretrigger disabled.
	ADC_SW_PRETRIGGER_0 = 0x04U	SW pretrigger 0.
	ADC_SW_PRETRIGGER_1 = 0x05U	SW pretrigger 1.
	ADC_SW_PRETRIGGER_2 = 0x06U	SW pretrigger 2.
	ADC_SW_PRETRIGGER_3 = 0x07U	SW pretrigger 3.

2.4.2.12 Adc_CmpChannelType

Enumeration	Adc_CmpChannelType	
Description	Select the channel compare mode.	
Values	Value	Description
	ADC_CMP_CHANNEL_ALL = 0U	Compare enabled in all channels.
	ADC_CMP_CHANNEL_SINGLE = 1U	Compare enabled in the specified channel.

2.4.3 Enumerations in Adc_Ptimer_Hw_Types.h

2.4.3.1 Ptimer_LoadValueModeType

Enumeration	Ptimer_LoadValueModeType	
Description	Ptimer value load mode.	
Values	Value	Description
	PTIMER_LOAD_VAL_IMMEDIATEL = 0U	Loaded immediately after load operation.
	PTIMER_LOAD_VAL_AT_MODULO_COUNTER = 1U	Loaded when counter hits the max count after load operation.
	PTIMER_LOAD_VAL_AT_NEXT_TRIGGER = 2U,	Loaded when detecting an input trigger after load operation.
	PTIMER_LOAD_VAL_AT_MODULO_COUNTER_OR_NEXT_TRIGGER = 3U	Loaded when counter hits the max count or detecting an input trigger after load operation.

2.4.3.2 Ptimer_ClockPreDividerType

Enumeration	Ptimer_ClockPreDividerType	
Description	Ptimer clock pre-divider factor.	
Values	Value	Description
	PTIMER_PRE_DIVIDE_BY_1 = 0U	The clock pre-divide factor is 1
	PTIMER_PRE_DIVIDE_BY_2 = 1U	The clock pre-divide factor is 2
	PTIMER_PRE_DIVIDE_BY_4 = 2U	The clock pre-divide factor is 4
	PTIMER_PRE_DIVIDE_BY_8 = 3U	The clock pre-divide factor is 8
	PTIMER_PRE_DIVIDE_BY_16 = 4U	The clock pre-divide factor is 16
	PTIMER_PRE_DIVIDE_BY_32 = 5U	The clock pre-divide factor is 32
	PTIMER_PRE_DIVIDE_BY_64 = 6U	The clock pre-divide factor is 64
	PTIMER_PRE_DIVIDE_BY_128 = 7U	The clock pre-divide factor is 128

2.4.3.3 Ptimer_ClockPreDivMultiplyFactorType

Enumeration	Ptimer_ClockPreDivMultiplyFactorType	
Description	Ptimer clock divider multiplication factor.	
Values	Value	Description
	PTIMER_PRE_DIVIDER_MULTIPLY_BY_1 = 0U	The clock multiplication factor for clock divider is 1.
	PTIMER_PRE_DIVIDER_MULTIPLY_BY_10 = 1U	The clock multiplication factor for clock divider is 10.
	PTIMER_PRE_DIVIDER_MULTIPLY_BY_20 = 2U	The clock multiplication factor for clock divider is 20.
	PTIMER_PRE_DIVIDER_MULTIPLY_BY_40 = 3U	The clock multiplication factor for clock divider is 40.

2.4.3.4 Ptimer_TrgSrcType

Enumeration	Ptimer_TrgSrcType	
Description	Select the Ptimer trigger source.	
Values	Value	Description
	PTIMER_TRGSRG_TRGSEL = 0x00U	Ptimer trigger source from TrgSel.
	PTIMER_TRGSRG_SW = 0x0FU	Ptimer trigger source from software trigger.

2.5 Typedefs

2.5.1 Typedefs in Adc_Types.h

- typedef void (***Adc_NotifyType**)(void)
Notification function pointer definition.

- typedef uint16 **Adc_ChannelIndexType**
Number of channels.
- typedef uint8 **Adc_HwUnitType**
Numeric ID of an ADC HW Unit.
- typedef uint8 **Adc_ClockSourceType**
Clock source for ADC.
- typedef uint16 **Adc_VoltageSourceType**
Reference voltage source.
- typedef uint16 **Adc_QueueIndexType**
Index for the queue of groups.
- typedef uint8 **Adc_ChannelType**
Numeric ID of an ADC channel.
- typedef uint16 **Adc_GroupType**
Numeric ID of an ADC channel group. No uC dependent.
- typedef uint8 **Adc_ResolutionType**
channel resolution in number of bits.
- typedef uint8 **Adc_GroupPriorityType**
ADC Channel group priority.
- typedef Adc_ChannelType **Adc_GroupDefType**
Definition of channels in a group.
- typedef uint16 **Adc_ValueGroupType**
Type for ADC result of one channel.
- typedef uint8 **Adc_PrescaleType**
Clock prescaler factor.
- typedef uint16 **Adc_ConversionTimeType**
Conversion time.

- typedef uint8 **Adc_SamplingTimeType**
Sampling time.
- typedef uint8 **Adc_SampleTimeOptType**
Sample time options.
- typedef uint16 **Adc_StreamNumSampleType**
Number of samples of a streaming conversion buffer.
- typedef uint16 **Adc_HwTriggerTimerType**
Type for the reload value of the ADC embedded timer

2.6 Structures

2.6.1 Adc_ConfigType

Structure	Adc_ConfigType
Description	Structure for Configuration data. Data structure containing the set of configuration parameters required for initializing the ADC Driver and ADC HW Unit(s).
Diagram	<pre> graph LR Adc_ConfigType((Adc_ConfigType)) --> pAdc((Adc_HwUnitConfigurationType *pAdc)) Adc_ConfigType --> pGroups((Adc_GroupConfigurationType *pGroups)) Adc_ConfigType --> pChannels((Adc_ChannelConfigurationType **pChannels)) Adc_ConfigType --> GroupIndex((Adc_GroupType *GroupIndex)) Adc_ConfigType --> HwUnitCount((Adc_HwUnitType HwUnitCount)) Adc_ConfigType --> GroupCount((Adc_GroupType GroupCount)) Adc_ConfigType --> CoreId((uint8 CoreId)) </pre>
Data Fields	• const Adc_HwUnitConfigurationType *pAdc HW unit configurations. • const Adc_GroupConfigurationType *pGroups Group configurations. • const Adc_ChannelConfigurationType *const *pChannels Channel configurations. • const Adc_GroupType *GroupIndex Group index to logical group map. • Adc_HwUnitType HwUnitCount Total number of used hardware units. • Adc_GroupType GroupCount Total number of groups.

	uint8 CoreId CoreId of the configuration.
--	---

2.6.2 Adc_HwUnitConfigurationType

Structure	Adc_HwUnitConfigurationType
Description	Structure for ADC hardware unit configuration.
Diagram	N/A
Data Fields	- const Adc_HwUnitType AdcHardwareUnitId ADC hardware unit ID. - const uint32 u32AdcCfg1Register Main configuration of CFG1 registers - const uint32 u32AdcCfg2Register Main configuration of CFG2 registers. - const uint32 u32AdcSmprRegister Main configuration of Sample Time registers. - const uint32 u32AdcIntEnRegister Main configuration of INT_ENABLE registers. - const Adc_PrescaleType PreDiv Optional ADC module specific clock prescale factor. - const Adc_ResolutionType Res Resolution of the ADC hardware. - const uint32 u32AdcPtimerClockSettings Settings for PRESCALER and MULT fields of PTIMER_STATUS_CTRL register. - const Adc_NotifyType pPtimerNotification Pointer to Ptimer notification function.

2.6.3 Adc_GroupConfigurationType

Structure	Adc_GroupConfigurationType
Description	Structure for group configuration.
Diagram	N/A
Data Fields	- const Adc_GroupType AdcGroupId Adc group id. - const Adc_HwUnitType HwUnit HW unit to which the group belongs to. - const Adc_GroupAccessModeType eAccessMode Access mode (Single/Streaming). - const Adc_GroupConvModeType eMode Conversion mode (OneShot/Continuous). - const Adc_GroupPriorityType Priority Priority of group. - const Adc_GroupReplacementType eReplacementMode Replacement mode.

	• <code>const Adc_TriggerSourceType eTriggerSource</code> HW/SW trigger. • <code>const Adc_HwTriggerSignalType eTriggerEdge</code> Hardware trigger edge. • <code>const Adc_TransferType eTransferType</code> Result data transfer method. • <code>const uint8 u8DmaChannel</code> DMA channel associated with the Adc group. • <code>const Adc_NotifyType Notification</code> Pointer to notification function. • <code>const Adc_StreamBufferModeType eBufferMode</code> Buffer Mode. • <code>const Adc_StreamNumSampleType NumSamples</code> Number of samples. • <code>const Adc_GroupDefType *pAssignment</code> Assigned channels to group. • <code>const uint16 *pDelay</code> Assigned channel delays to group. • <code>const Adc_ChannelIndexType AssignedChannelCount</code> Number of channels. • <code>const boolean bEnableHwAvg</code> Whether to enable hardware average. • <code>const Adc_AverageType u32AvgSelect</code> Hardware average numbers. • <code>const uint8 u8AdcGroupBackToBack</code> Enables or Disables the ADC Back to Back configuration for Group Channels. • <code>const uint8 u8AdcGroupChannelDelays</code> Enables or Disables the ADC pre-trigger delay configuration for each Group Channel. • <code>const uint16 u16PtimerDelay</code> The delay for next Ptimer. • <code>const boolean bUseContinuousPeriod;</code> Whether to use the Ptimer period for continuous mode • <code>const uint16 u16PtimerPeriod</code> The period Ptimer for continuous mode.
--	---

2.6.4 Adc_ChannelConfigurationType

Structure	<code>Adc_ChannelConfigurationType</code>
Description	Structure for channel configuration.
Diagram	N/A
Data Fields	• <code>const Adc_ChannelType ChId</code> Channel ID. • <code>const Adc_ResolutionType Res</code> Resolution of the ADC HW units.

	• const Adc_SampleTimeOptType SampleTimeOpt Sample time option of the ADC channel. • const boolean bAdcChannelLimitcheck Enables or disables the usage of limit checking for the ADC channel. • const Adc_ChannelRangeSelectType eChannelRange Conversion range selection. • const Adc_ValueGroupType ChannelHighLimit High limit channel conversion value. • const Adc_ValueGroupType ChannelLowLimit Low limit channel conversion value.
--	--

2.6.5 Adc_UnitStatusType

Structure	Adc_UnitStatusType
Description	Structure for hardware unit status.
Diagram	N/A
Data Fields	• volatile Adc_QueueIndexType SwNormalQueueIndex Filled slots in the queue. • volatile Adc_GroupType SwNormalQueue[ADC_QUEUE_MAX_DEPTH_MAX] Queued groups indexes, always executing Queue[0]. • volatile Adc_GroupType HwNormalQueue[ADC_HW_QUEUE] The depth of the hardware normal queue. • volatile Adc_QueueIndexType HwNormalQueueIndex Filled slots in the Hw normal queue. • uint8 u8ScUsed Used ADC_SCn registers.

2.6.6 Adc_GroupStatusType

Structure	Adc_GroupStatusType
Description	Structure for group status.
Diagram	N/A
Data Fields	• volatile Adc_StatusType eConversion Group status. • volatile Adc_GroupConversionStateType eAlreadyConverted Group was previously converted or not. • Adc_ValueGroupType *pResultsBuffer Pointer to user result buffer. • Adc_HwTriggeringType eHwTriggering HW trigger enabled/disabled. • Adc_NotificationType eNotification Notification enabled/disabled. • volatile Adc_StreamNumSampleType ResultIndex Index into streaming buffer that is currently being filled.

- `Adc_ChannelIndexType` `CurrentChannel`
Current channel in use.

2.7 API Functions

2.7.1 Functions in `Adc.h`

2.7.1.1 `void Adc_Init(const Adc_ConfigType *ConfigPtr)`

Function	<code>void Adc_Init(const Adc_ConfigType *ConfigPtr)</code>	
Description	Initializes the ADC hardware unit and the driver.	
Diagram	<pre> graph LR Adc_Init[Adc_Init] --> Adc_HL_Init[Adc_HL_Init] Adc_Init --> Adc_InitGroupsStatus[Adc_InitGroupsStatus] Adc_Init --> Adc_InitUnitStatus[Adc_InitUnitStatus] Adc_Init --> Adc_ReportDetError[Adc_ReportDetError] </pre>	
Parameters	Parameter	Description
	<code>pConfigPtr</code>	Pointer to configuration set in Variant PB (Variant PC requires a <code>NULL_PTR</code>).
Returns	N/A	

2.7.1.2 `Std_ReturnType Adc_SetupResultBuffer(Adc_GroupType Group, Adc_ValueGroupType *DataBufferPtr)`

Function	<code>Std_ReturnType Adc_SetupResultBuffer(Adc_GroupType Group, Adc_ValueGroupType *DataBufferPtr)</code>	
Description	Initializes the group specific ADC result buffer pointer as configured to point to the <code>pDataBufferPtr</code> address which is passed as parameter.	
Diagram	N/A	
Parameters	Parameter	Description
	<code>Group</code>	Numeric ID of requested ADC channel group.
	<code>DataBufferPtr</code>	Pointer to result data buffer.
Returns	<code>Std_ReturnType</code> <i>Standard return type.</i>	

2.7.1.3 `void Adc_DeInit(void)`

Function	<code>void Adc_DeInit(void)</code>
Description	Returns all ADC HW Units to a state comparable to their power on reset state.
Diagram	N/A
Parameters	N/A
Returns	N/A

2.7.1.4 `void Adc_StartGroupConversion( Adc_GroupType Group )`

Function	<code>void Adc_StartGroupConversion(Adc_GroupType Group)</code>
Description	Starts the conversion of all channels of the requested ADC Channel group.
Diagram	N/A

Parameters	Parameter	Description
	Group	Numeric ID of requested ADC channel group.
Returns	N/A	

2.7.1.5 void Adc_StopGroupConversion (Adc_GroupType Group)

Function	void Adc_StopGroupConversion (Adc_GroupType Group)	
Description	Stops the conversion of all channels of the requested ADC Channel group.	
Diagram	N/A	
Parameters	Parameter	Description
	Group	Numeric ID of requested ADC channel group.
Returns	N/A	

2.7.1.6 Std_ReturnType Adc_ReadGroup(Adc_GroupType Group,Adc_ValueGroupType *DataBufferPtr)

Function	Std_ReturnType Adc_ReadGroup(Adc_GroupType Group,Adc_ValueGroupType *DataBufferPtr)	
Description	Reads the group conversion results of the last completed conversion round of the requested group and stores the channel values starting at the pDataBufferPtr address. The group channel values are stored in ascending channel number order (in contrast to the storage layout of the result buffer if streaming access is configured).	
Diagram	N/A	
Parameters	Parameter	Description
	Group	Numeric ID of requested ADC channel group.
	pDataBufferPtr	ADC results of all channels of the selected group are stored in the data buffer addressed with the pointer.
Returns	Std_ReturnType Standard return type.	

2.7.1.7 void Adc_EnableHardwareTrigger (Adc_GroupType Group)

Function	void Adc_EnableHardwareTrigger (Adc_GroupType Group)	
Description	Enables the hardware trigger for the requested ADC Channel group.	
Diagram	N/A	
Parameters	Parameter	Description
	Group	Numeric ID of requested ADC channel group.
Returns	N/A	

2.7.1.8 void Adc_DisableHardwareTrigger (Adc_GroupType Group)

Function	void Adc_DisableHardwareTrigger (Adc_GroupType Group)	
Description	Disables the hardware trigger for the requested ADC Channel group.	
Diagram	N/A	
Parameters	Parameter	Description
	Group	Numeric ID of requested ADC channel group.
Returns	N/A	

2.7.1.9 void Adc_EnableGroupNotification (Adc_GroupType Group)

Function	void Adc_EnableGroupNotification (Adc_GroupType Group)	
Description	Enables the notification mechanism for the requested ADC channel group.	
Diagram	N/A	
Parameters	Parameter	Description
	Group	Numeric ID of requested ADC channel group.
Returns	N/A	

2.7.1.10 void Adc_DisableGroupNotification (Adc_GroupType Group)

Function	void Adc_DisableGroupNotification (Adc_GroupType Group)	
Description	Disables the notification mechanism for the requested ADC channel group.	
Diagram	N/A	
Parameters	Parameter	Description
	Group	Numeric ID of requested ADC channel group.
Returns	N/A	

2.7.1.11 Adc_StatusType Adc_GetGroupStatus (Adc_GroupType Group)

Function	Adc_StatusType Adc_GetGroupStatus (Adc_GroupType Group)	
Description	This function will return the conversion status of the requested ADC channel group.	
Diagram	N/A	
Parameters	Parameter	Description
	Group	Numeric ID of requested ADC channel group.
Returns	Adc_StatusType Conversion status for the requested group.	

2.7.1.12 Adc_StreamNumSampleType Adc_GetStreamLastPointer(Adc_GroupType Group, Adc_ValueGroupType **PtrToSamplePtr)

Function	Adc_StreamNumSampleType Adc_GetStreamLastPointer(Adc_GroupType Group, Adc_ValueGroupType **PtrToSamplePtr)	
Description	Returns the number of valid samples per channel, stored in the result buffer. Reads a pointer, pointing to a position in the group result buffer. With the pointer position, the results of all group channels of the last completed conversion round can be accessed. With the pointer and the return value, all valid group conversion results can be accessed (the user has to take the layout of the result buffer into account).	
Diagram	N/A	
Parameters	Parameter	Description
	Group	Numeric ID of requested ADC channel group.
	PtrToSamplePtr	Pointer to result buffer pointer.
Returns	Adc_StreamNumSampleType Number of valid samples per channel.	

2.7.1.13 void Adc_GetVersionInfo (Std_VersionInfoType * versioninfo)

Function	void Adc_GetVersionInfo (Std_VersionInfoType * versioninfo)
Description	Returns the version information of this module.

Diagram	N/A	
Parameters	Parameter	Description
	versioninfo	Pointer to where to store the version information of this module.
Returns	N/A	

2.8 Hardware Functions

2.8.1 Functions in Adc_Hw.h

2.8.1.1 void Adc_HL_Init (const Adc_ConfigType * pCfgPtr)

Function	void Adc_HL_Init (const Adc_ConfigType * pCfgPtr)	
Description	This function initializes the ADC hardware module, and the ADC driver.	
Parameters	Parameter	Description
	pCfgPtr	Configuration data pointer.
Returns	N/A	
Referenced By	Adc_Init()	

2.8.1.2 void Adc_HL_DeInit (const uint8 CoreId)

Function	void Adc_HL_DeInit (const uint8 CoreId)	
Description	This function deinitializes the ADC driver, and ensures that the ADC hardware will be in the power on after reset state.	
Parameters	Parameter	Description
	CoreId	The core ID of the caller.
Returns	N/A	
Referenced By	Adc_DeInit()	

2.8.1.3 void Adc_HL_StartConversion (const Adc_HwUnitType Unit, const Adc_GroupType Group, const uint8 CoreId)

Function	void Adc_HL_StartConversion (const Adc_HwUnitType Unit, const Adc_GroupType Group, const uint8 CoreId)	
Description	This function update queue before starting a group conversion.	
Parameters	Parameter	Description
	Unit	The hardware Unit.
	Group	The group ID.
	CoreId	The core ID of the caller.
Returns	N/A	
Referenced By	Adc_StartGroupConversion()	

2.8.1.4 void Adc_HL_StopConversion (const Adc_HwUnitType Unit, const Adc_GroupType Group, const uint8 CoreId)

Function	void Adc_HL_StopConversion (const Adc_HwUnitType Unit, const Adc_GroupType Group, const uint8 CoreId)	
Description	This function updates queue before stopping a group conversion.	
Parameters	Parameter	Description

	Unit	The hardware Unit.
	Group	The group ID.
	CoreId	The core ID of the caller.
Returns	N/A	
Referenced By	Adc_StopGroupConversion()	

2.8.1.5 void Adc_HL_EnableHardwareTrigger (const Adc_HwUnitType Unit, const Adc_GroupType Group, const uint8 CoreId)

Function	void Adc_HL_EnableHardwareTrigger (const Adc_HwUnitType Unit, const Adc_GroupType Group, const uint8 CoreId)	
Description	This function updates the group status before enabling the hardware trigger.	
Parameters	Parameter	Description
	Unit	The hardware Unit.
	Group	The group ID.
	CoreId	The core ID of the caller.
Returns	N/A	
Referenced By	Adc_EnableHardwareTrigger()	

2.8.1.6 void Adc_HL_DisableHardwareTrigger (const Adc_HwUnitType Unit, const Adc_GroupType Group, const uint8 CoreId)

Function	void Adc_HL_DisableHardwareTrigger (const Adc_HwUnitType Unit, const Adc_GroupType Group, const uint8 CoreId)	
Description	This function updates the group status before disabling the hardware trigger.	
Parameters	Parameter	Description
	Unit	The hardware Unit.
	Group	The group id.
	CoreId	The core ID of the caller.
Returns	N/A	
Referenced By	Adc_DisableHardwareTrigger()	

2.8.1.7 void Adc_HL_UpdateGroupStatusWithoutInterrupt (const Adc_GroupType Group, const uint8 CoreId)

Function	void Adc_HL_UpdateGroupStatusWithoutInterrupt (const Adc_GroupType Group, const uint8 CoreId)	
Description	When the interrupt is not enabled, calling of Adc_GetGroupStatus will trigger a group status update, this function is responsible for updating the group status.	
Parameters	Parameter	Description
	Group	The group ID.
	CoreId	The core ID of the caller.
Returns	N/A	
Referenced By	Adc_GetGroupStatus()	

2.8.1.8 Std_ReturnType Adc_HL_ReadGroup (const Adc_GroupType Group, const uint8 CoreId, Adc_ValueGroupType * pDataPtr)

Function	Std_ReturnType Adc_HL_ReadGroup (const Adc_GroupType Group, const uint8 CoreId, Adc_ValueGroupType * pDataPtr)	
Description	This function reads the group conversion data and updates the group status.	
Parameters	Parameter	Description
	Group	The group ID.
	CoreId	The core ID of the caller.
	pDataPtr	Pointer to a buffer that will be filled by the conversion results.
Returns	Std_ReturnType E_OK or E_NOT_OK.	
Referenced By	Adc_ReadGroup	

2.8.1.9 void Adc_HL_EndPartialConversion(const Adc_HwUnitType Unit, const Adc_GroupType Group, const uint8 CoreId)

Function	void Adc_HL_EndPartialConversion(const Adc_HwUnitType Unit, const Adc_GroupType Group, const uint8 CoreId)	
Description	When a normal (not injected) conversion ends, and the end of conversion ISR is called, this function takes care about the driver and the module state transitions.	
Parameters	Parameter	Description
	Unit	The hardware Unit.
	Group	The group ID.
	CoreId	The core ID of the caller.
Returns	N/A	
Referenced By	Adc_ISR_EndGroupConversion()	

2.8.2 Functions in Adc_Lld.h

2.8.2.1 void Adc_LL_InitUnitHardware (const Adc_HwUnitType Unit, const Adc_HwUnitConfigurationType * pConfigPtr)

Function	void Adc_LL_InitUnitHardware (const Adc_HwUnitType Unit, const Adc_HwUnitConfigurationType * pConfigPtr)	
Description	Initializes the ADC hardware as per configuration.	
Parameters	Parameter	Description
	Unit	The hardware Unit.
	pConfigPtr	Pointer to the hardware configuration structure to be used.
Returns	N/A	
Referenced By	Adc_HL_Init()	

2.8.2.2 Adc_LL_DeInitUnitHardware (const Adc_HwUnitType Unit, const uint8 CoreId)

Function	Adc_LL_DeInitUnitHardware (const Adc_HwUnitType Unit, const uint8 CoreId)	
Description	This function deinitializes the ADC hardware.	
Parameters	Parameter	Description
	Unit	The hardware Unit.

	CoreId	The core ID of the caller.
Returns	N/A	
Referenced By	Adc_HL_DeInit()	

2.8.2.3 void Adc_LL_StartHwTrigConversion (const Adc_HwUnitType Unit, const Adc_GroupType Group, const uint8 CoreId)

Function	void Adc_LL_StartHwTrigConversion (const Adc_HwUnitType Unit, const Adc_GroupType Group, const uint8 CoreId)	
Description	This function performs starting a hardware trigger conversion.	
Parameters	Parameter	Description
	Unit	The hardware Unit.
	Group	The group of which conversion will be started.
	CoreId	The core ID of the caller.
Returns	N/A	
Referenced By	Adc_HL_UpdateStatusGetData(), Adc_LL_EnableHardwareTrigger()	

2.8.2.4 void Adc_LL_EnableHardwareTrigger (const Adc_HwUnitType Unit, const Adc_GroupType Group, const uint8 CoreId)

Function	void Adc_LL_EnableHardwareTrigger (const Adc_HwUnitType Unit, const Adc_GroupType Group, const uint8 CoreId)	
Description	This function enables the hardware triggers for the specified group.	
Parameters	Parameter	Description
	Unit	The hardware Unit. (on other platforms there are more HW units)
	Group	The group for which the hardware triggers will be enabled.
	CoreId	The core ID of the caller.
Returns	N/A	
Referenced By	Adc_HL_EnableHardwareTrigger()	

2.8.2.5 void Adc_LL_DisableHardwareTrigger (const Adc_HwUnitType Unit, const Adc_GroupType Group, const uint8 CoreId)

Function	void Adc_LL_DisableHardwareTrigger (const Adc_HwUnitType Unit, const Adc_GroupType Group, const uint8 CoreId)	
Description	This function disables the hardware triggers for the specified group.	
Parameters	Parameter	Description
	Unit	The hardware Unit. (on other platforms there are more HW units)
	Group	The group for which the hardware triggers will be enabled.
	CoreId	The core ID of the caller.
Returns	N/A	
Referenced By	Adc_HL_DisableHardwareTrigger()	

2.8.2.6 void Adc_LL_StartNormalConversion (const Adc_HwUnitType Unit, const uint8 CoreId)

Function	void Adc_LL_StartNormalConversion (const Adc_HwUnitType Unit, const uint8 CoreId)	
Description	This function starts the conversion on the specified hardware Unit. This function should be called with the ADC Unit on standby (no ADC interruptions allowed because its code is not protected by critical regions) - no conversions must be running.	
Parameters	Parameter	Description
	Unit	The hardware Unit.
	CoreId	The core ID of the caller.
Returns	N/A	
Referenced By	Adc_HL_StartConversion(), Adc_HL_StopConversion(), Adc_HL_UpdateSwQueue()	

2.8.2.7 void Adc_LL_ConfigurePartialConversion (const Adc_HwUnitType Unit, const Adc_GroupType Group, const uint8 CoreId, const Adc_TriggerSourceType eTrigger, const uint8 u8NumChannel)

Function	void Adc_LL_ConfigurePartialConversion (const Adc_HwUnitType Unit, const Adc_GroupType Group, const uint8 CoreId, const Adc_TriggerSourceType eTrigger, const uint8 u8NumChannel)	
Description	This function configures channels of a group if the number of channels in the group exceeds the maximum channels allowed for a conversion, a part of channels of the group (= max allowed channels) will be configured in start conversion in the first time. The remaining channels of the group will be separated into other parts which are configured and converted in group interrupt.	
Parameters	Parameter	Description
	Unit	ADC Unit ID.
	Group	The group number.
	CoreId	The core ID of the caller.
	eTrigger	Conversion trigger source.
	u8NumChannel	Number of channels to be configured.
Returns	N/A	
Referenced By	Adc_HL_EndPartialConversion(), Adc_LL_StartNormalConversion(), Adc_LL_StartHwTrigConversion(), Adc_LL_RestartContinuousConversion()	

2.8.2.8 void Adc_LL_StopCurrentConversion (const Adc_HwUnitType Unit, const Adc_GroupType Group)

Function	void Adc_LL_StopCurrentConversion (const Adc_HwUnitType Unit, const Adc_GroupType Group)	
Description	This function stops the current conversion of a group.	
Parameters	Parameter	Description
	Unit	The hardware Unit.
	Group	ADC group number.
Returns	N/A	
Referenced By	Adc_HL_StartConversion(), Adc_HL_StopConversion(), Adc_HL_EndPartialConversion()	

2.8.2.9 Std_ReturnType Adc_LL_WaitConversionStopped (const Adc_HwUnitType Unit)

Function	Std_ReturnType Adc_LL_WaitConversionStopped (const Adc_HwUnitType Unit)	
Description	This function waits the current conversion to stop.	
Parameters	Parameter	Description

	Unit	The hardware Unit.
Returns	Std_ReturnType E_OK or E_NOT_OK.	
Referenced By	Adc_HL_StartConversion(), Adc_HL_StopConversion(), Adc_HL_EndPartialConversion()	

2.8.2.10 void Adc_LL_RestartContinuousConversion (const Adc_HwUnitType Unit, const Adc_GroupType Group, const uint8 CoreId)

Function	void Adc_LL_RestartContinuousConversion (const Adc_HwUnitType Unit, const Adc_GroupType Group, const uint8 CoreId)	
Description	This function restarts conversion in case of streaming access mode is enabled.	
Parameters	Parameter	Description
	Unit	The hardware Unit.
	Group	ADC group number.
	CoreId	The core ID of the caller.
Returns	N/A	
Referenced By	Adc_HL_UpdateSwQueue()	

2.8.2.11 Std_ReturnType Adc_LL_CheckConversionSequenceStatus (const Adc_HwUnitType Unit, const Adc_GroupType Group, const uint8 CoreId)

Function	boolean Adc_LL_CheckConversionSequenceStatus (const Adc_HwUnitType Unit, const Adc_GroupType Group, const uint8 CoreId)	
Description	This function checks whether the conversion sequence is completed	
Parameters	Parameter	Description
	Unit	The hardware Unit.
	Group	ADC group number.
	CoreId	The core ID of the caller.
Returns	boolean Whether the conversion is completed	
Referenced By	Adc_HL_UpdateGroupStatusWithoutInterrupt(), Adc_ISR_EndGroupConversion()	

2.8.2.12 Std_ReturnType Adc_LL_GetConversionSequenceResults (const Adc_HwUnitType Unit, const Adc_GroupType Group, const uint8 CoreId, Adc_ValueGroupType *pDataPtr)

Function	Std_ReturnType Adc_LL_GetConversionSequenceResults (const Adc_HwUnitType Unit, const Adc_GroupType Group, const uint8 CoreId, Adc_ValueGroupType *pDataPtr)	
Description	This function gets the results of the conversion sequence	
Parameters	Parameter	Description
	Unit	The hardware Unit.
	Group	ADC group number.
	CoreId	The core ID of the caller.
	pDataPtr	The pointer to the buffer which will be filled by the conversion results.
Returns	Std_ReturnType E_OK or E_NOT_OK.	
Referenced By	Adc_HL_UpdateGroupStatusWithoutInterrupt(), Adc_HL_EndPartialConversion()	

2.8.2.13 void Adc_LL_ReConfigureDma (const Adc_HwUnitType Unit, const Adc_GroupType Group, const uint8 CoreId)

Function	void Adc_LL_ReConfigureDma (const Adc_HwUnitType Unit, const Adc_GroupType Group, const uint8 CoreId)	
Description	This function reconfigures Dma when Ptimer sequence error occurred	
Parameters	Parameter	Description
	Unit	The hardware Unit.
	Group	ADC group number.
	CoreId	The core ID of the caller.
Returns	N/A	
Referenced By	Adc_Ptimer_ChannelSeqErr()	

2.8.3 Functions in Adc_Ptimer_Hw.h**2.8.3.1 void Adc_Ptimer_InitUnitHardware (Adc_HwUnitType Unit, uint32 u32PtimerClockConfig)**

Function	void Adc_Ptimer_InitUnitHardware (Adc_HwUnitType Unit, uint32 u32PtimerClockConfig)	
Description	This function initializes Ptimer clock configuration: prescaler, multiplication, channel sequence error	
Parameters	Parameter	Description
	Unit	The current HW Unit.
	u32PtimerClockConfig	Clock configuration for Ptimer
Returns	N/A	
Referenced By	Adc_LL_InitUnitHardware()	

2.8.3.2 void Adc_Ptimer_DeInitUnitHardware (Adc_HwUnitType Unit)

Function	void Adc_Ptimer_DeInitUnitHardware (Adc_HwUnitType Unit)	
Description	This function deinitializes the Ptimer hardware.	
Parameters	Parameter	Description
	Unit	The current HW Unit.
Returns	N/A	
Referenced By	Adc_LL_DeInitUnitHardware()	

2.8.3.3 void Adc_Ptimer_ConfigurePartialConversion (Adc_HwUnitType Unit, const Adc_GroupConfigurationType *pGroupPtr, Ptimer_TrgSrcType eTrigger, uint16 u16CurrentChannel, uint8 u8NumChannel)

Function	void Adc_Ptimer_ConfigurePartialConversion (Adc_HwUnitType Unit, const Adc_GroupConfigurationType *pGroupPtr, Ptimer_TrgSrcType eTrigger, uint16 u16CurrentChannel, uint8 u8NumChannel)	
Description	This function configures Ptimer hardware for each conversion.	
Parameters	Parameter	Description
	Unit	The current HW Unit.
	pGroupPtr	pointer to group configuration
	eTrigger	hardware or software trigger
	u16CurrentChannel	last channel converted
	u8NumChannel	number of channel used for this conversion

Returns	N/A
Referenced By	Adc_LL_ConfigurePartialConversion()

2.8.3.4 void Adc_Ptimer_StartSoftwareConversion (Adc_HwUnitType Unit)

Function	void Adc_Ptimer_StartSoftwareConversion (Adc_HwUnitType Unit)	
Description	This function starts software conversion via Ptimer.	
Parameters	Parameter	Description
	Unit	The current HW Unit.
Returns	N/A	
Referenced By	Adc_LL_RestartContinuousConversion()	

2.8.3.5 void Adc_Ptimer_SetPtimerMode (Adc_HwUnitType Unit, boolean bContinuous)

Function	void Adc_Ptimer_SetPtimerMode (Adc_HwUnitType Unit, boolean bContinuous)	
Description	This function changes mode of Ptimer hardware unit. This function changes Ptimer hardware unit to continuous trigger mode if ADC converts without interrupt.	
Parameters	Parameter	Description
	Unit	The current HW Unit.
	<i>bContinuous</i>	Whether to set the continuous flag.
Returns	N/A	
Referenced By	Adc_LL_StartNormalConversion()	

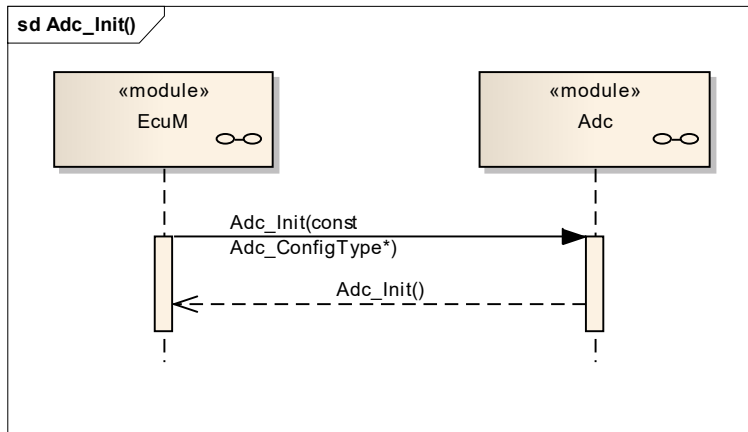
2.8.3.6 void Adc_Ptimer_StopConversion (Adc_HwUnitType Unit)

Function	void Adc_Ptimer_StopConversion (Adc_HwUnitType Unit)	
Description	This function disables Ptimer module and make control registers to reset value	
Parameters	Parameter	Description
	Unit	The current HW Unit.
Returns	N/A	
Referenced By	Adc_LL_StopConversionCheckTimeout()	

2.9 API Sequence Diagram

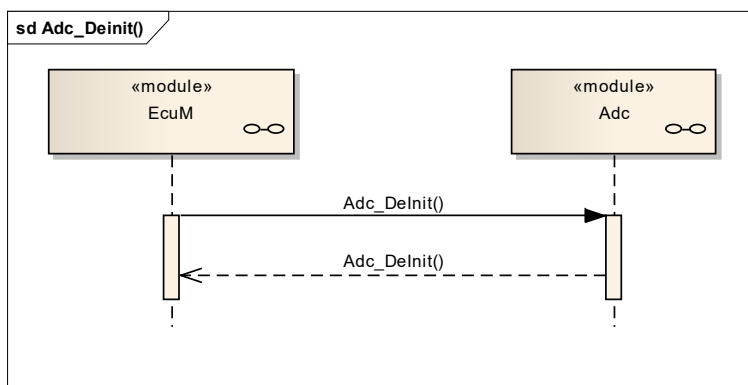
2.9.1 Initialization of the ADC Driver

The ECU State Manager (EcuM) is responsible for calling the initialization function.

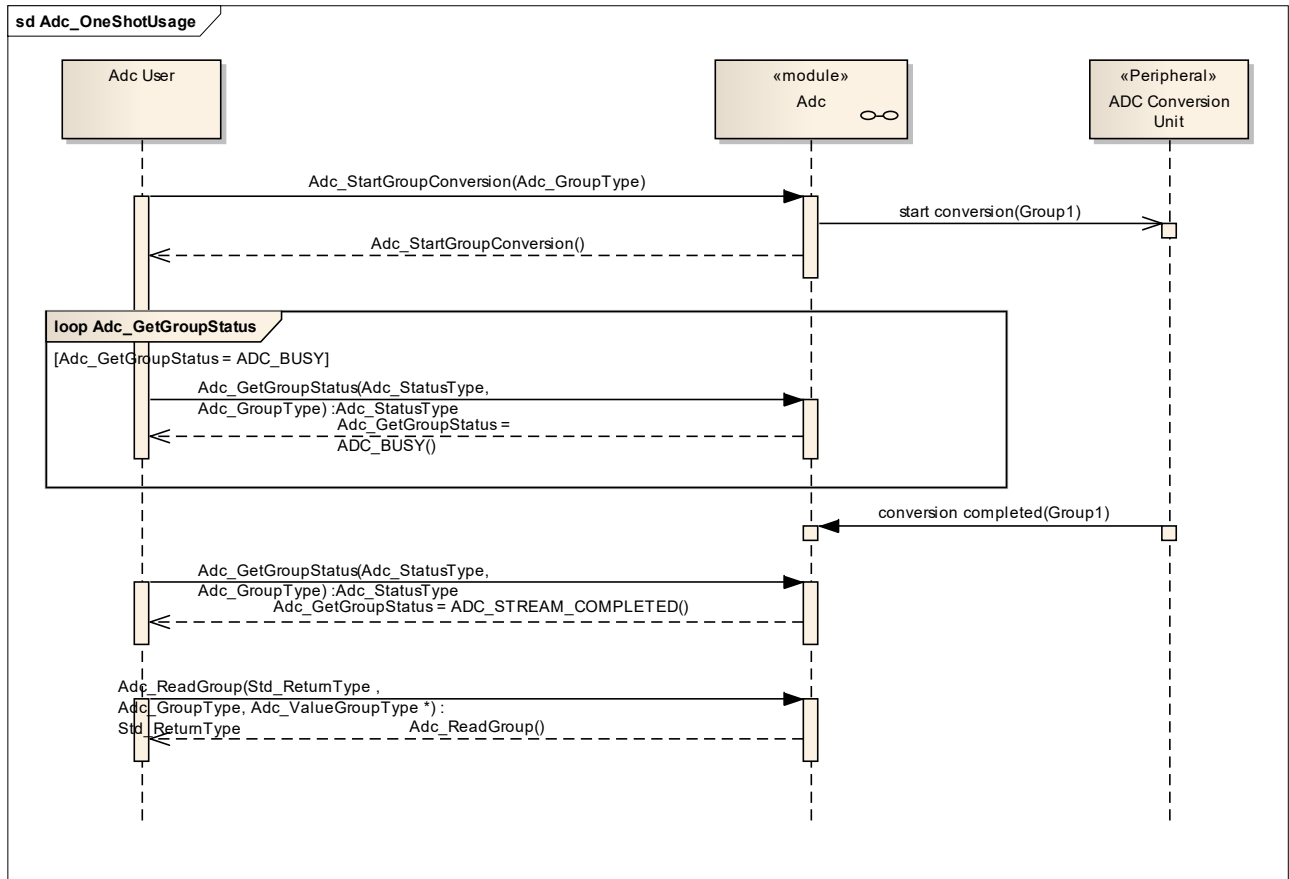


2.9.2 De-Initialization of the ADC Driver

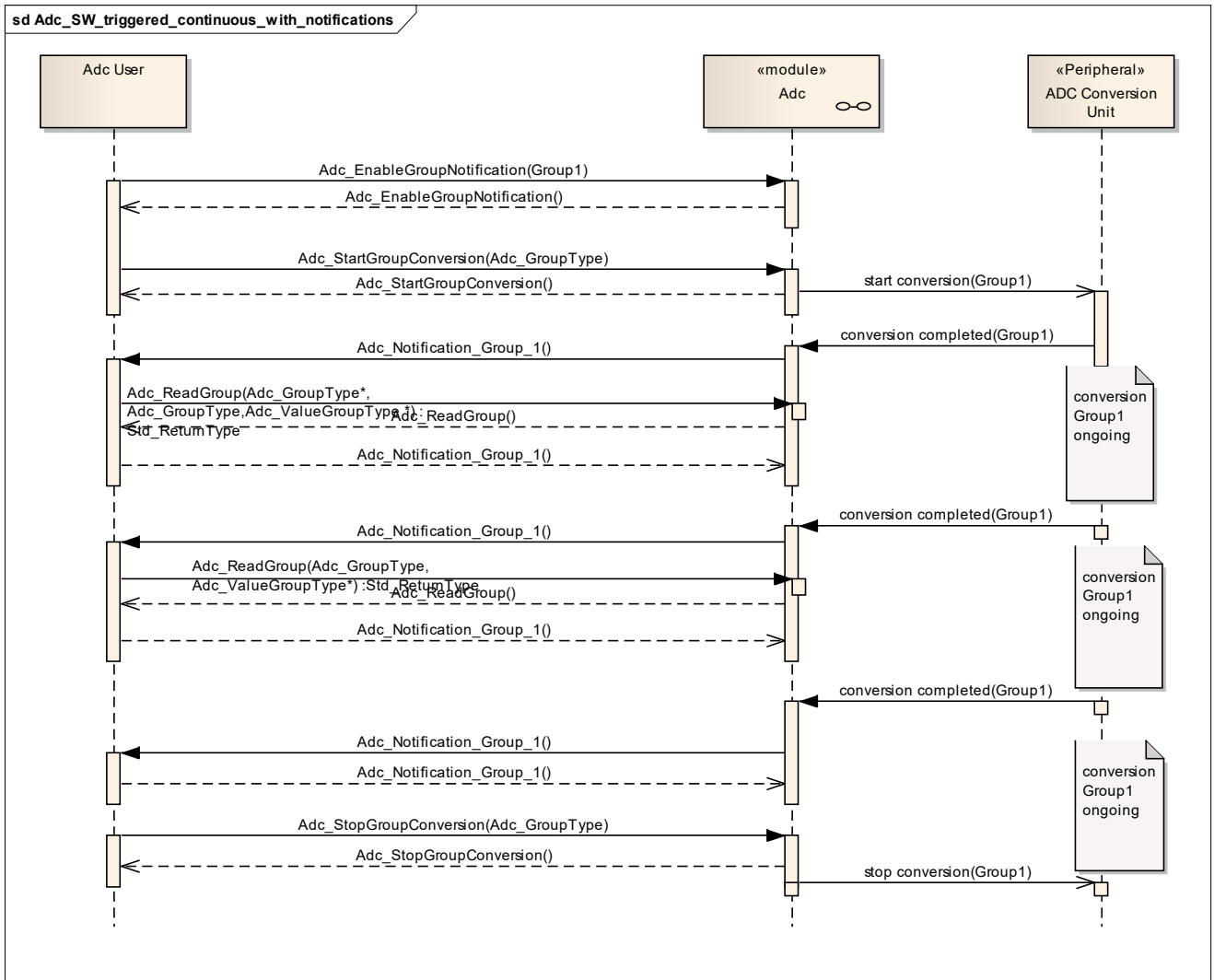
It's the responsibility of the HW design that this state doesn't lead to undefined activities in the μ c.



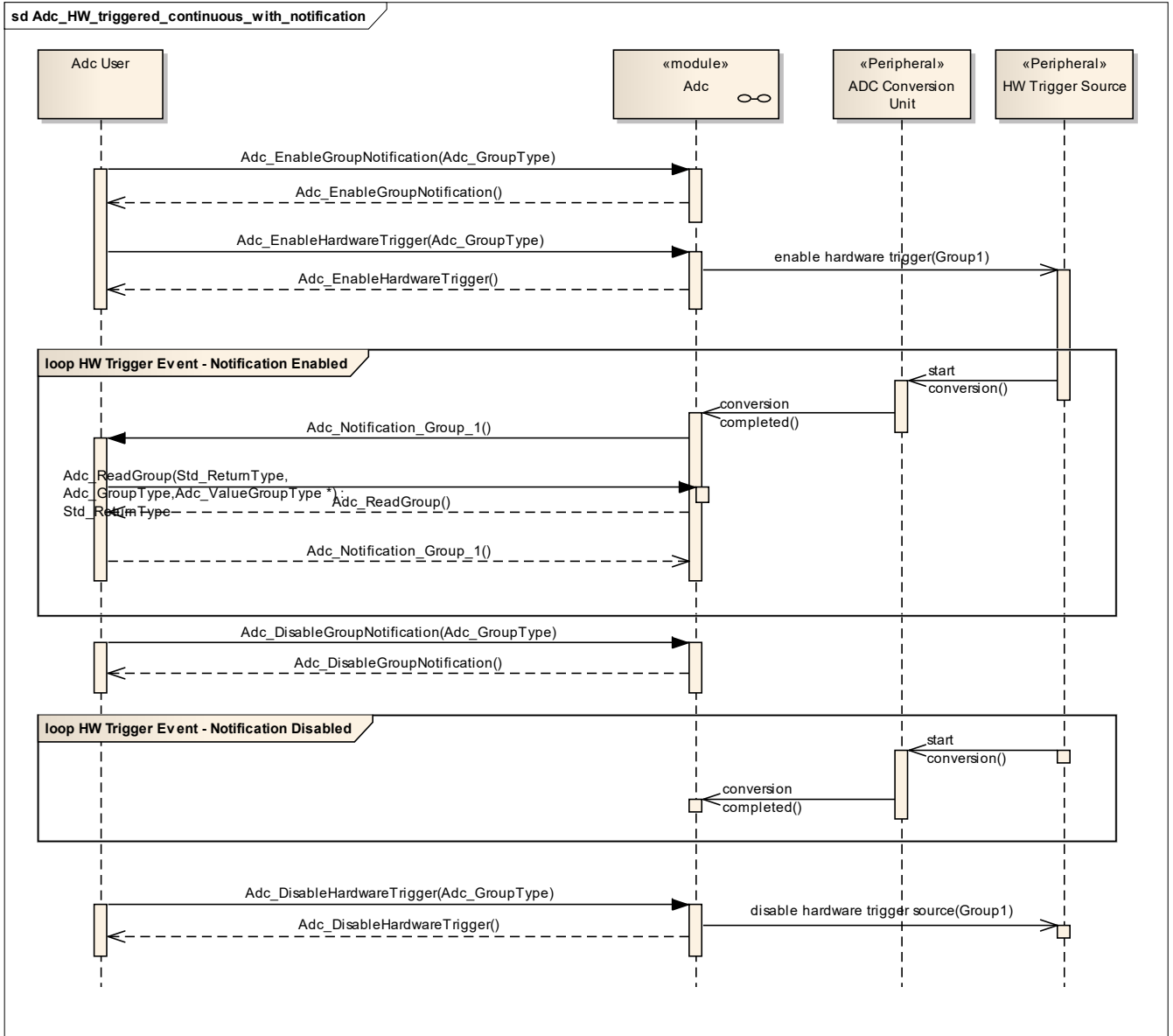
2.9.3 Software Triggered One-Shot Conversion without Notification



2.9.4 Software Triggered Continuous Conversion with Notification

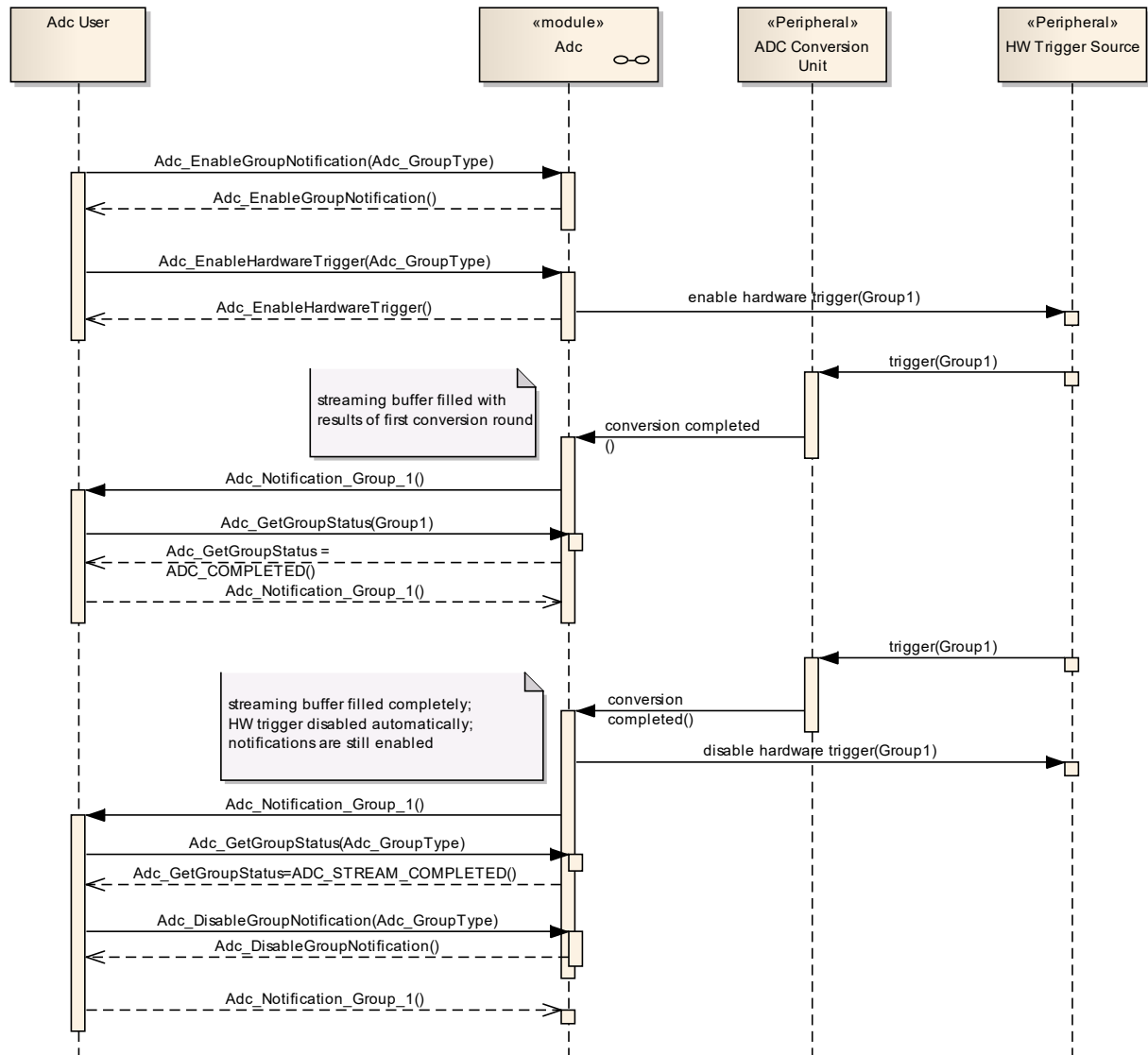


2.9.5 Hardware Triggered One-Shot Conversion with Notification

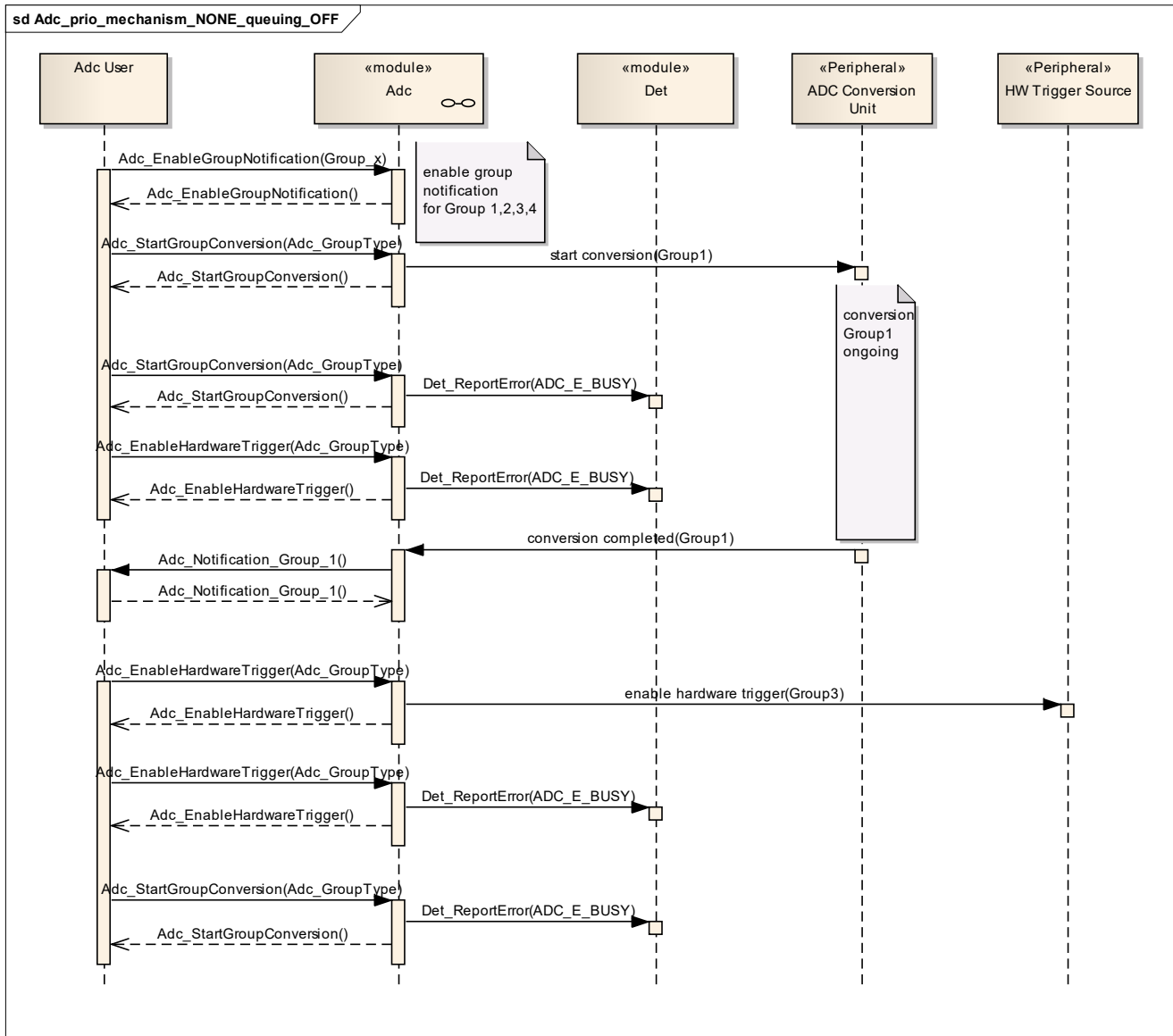


2.9.6 HW Trigger - One-Shot Conversion - Linear Streaming

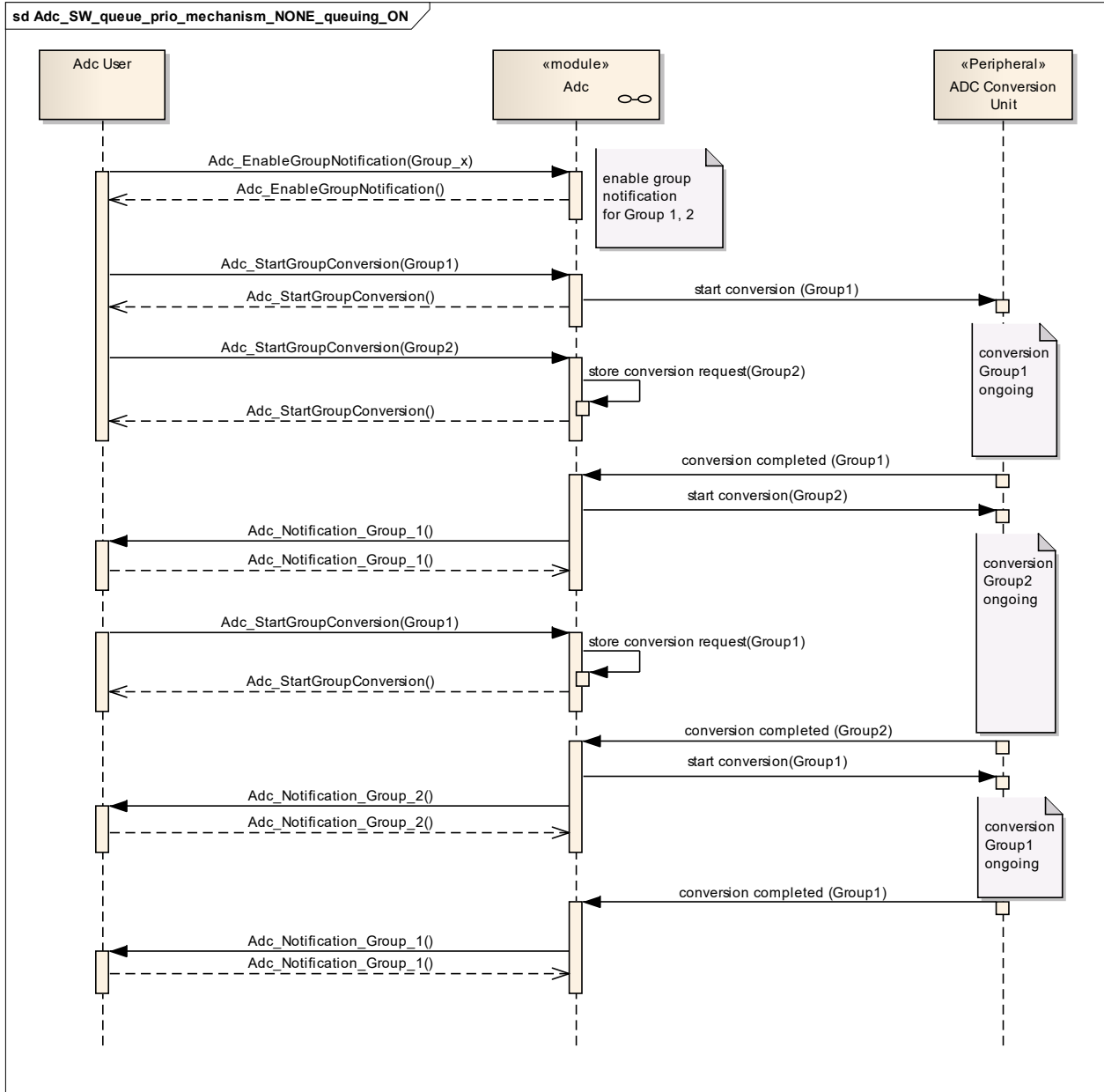
sd Adc_HW_triggered_oneshot_linear_streaming



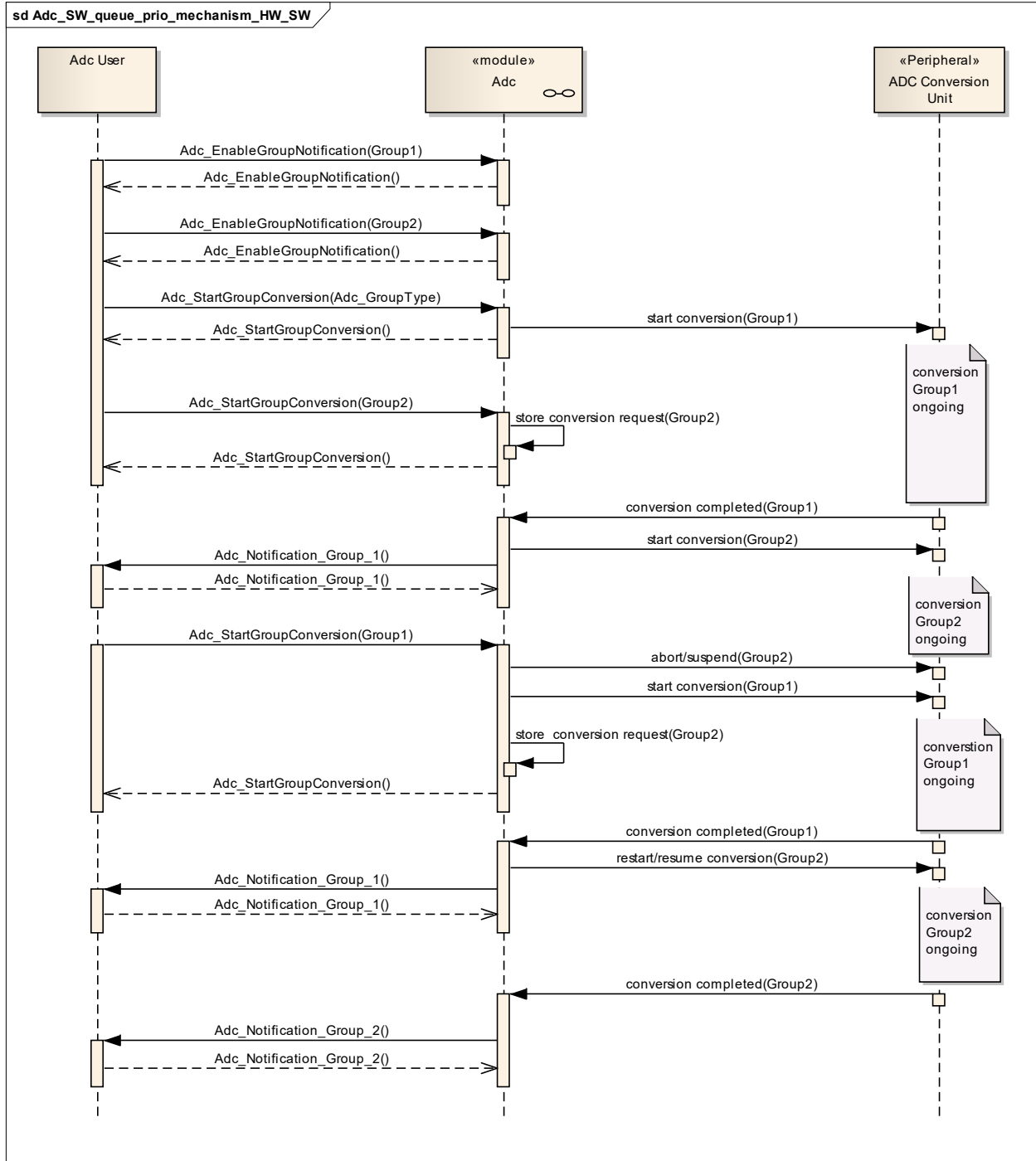
2.9.7 No Priority Mechanism – No Queuing



2.9.8 No Priority Mechanism – SW Queuing



2.9.9 HW_SW Priority Mechanism – SW Queuing



Chapter 3 Tresos Configuration Items

3.1 Container Inclusion Relation

The ADC module has the following 8 containers:

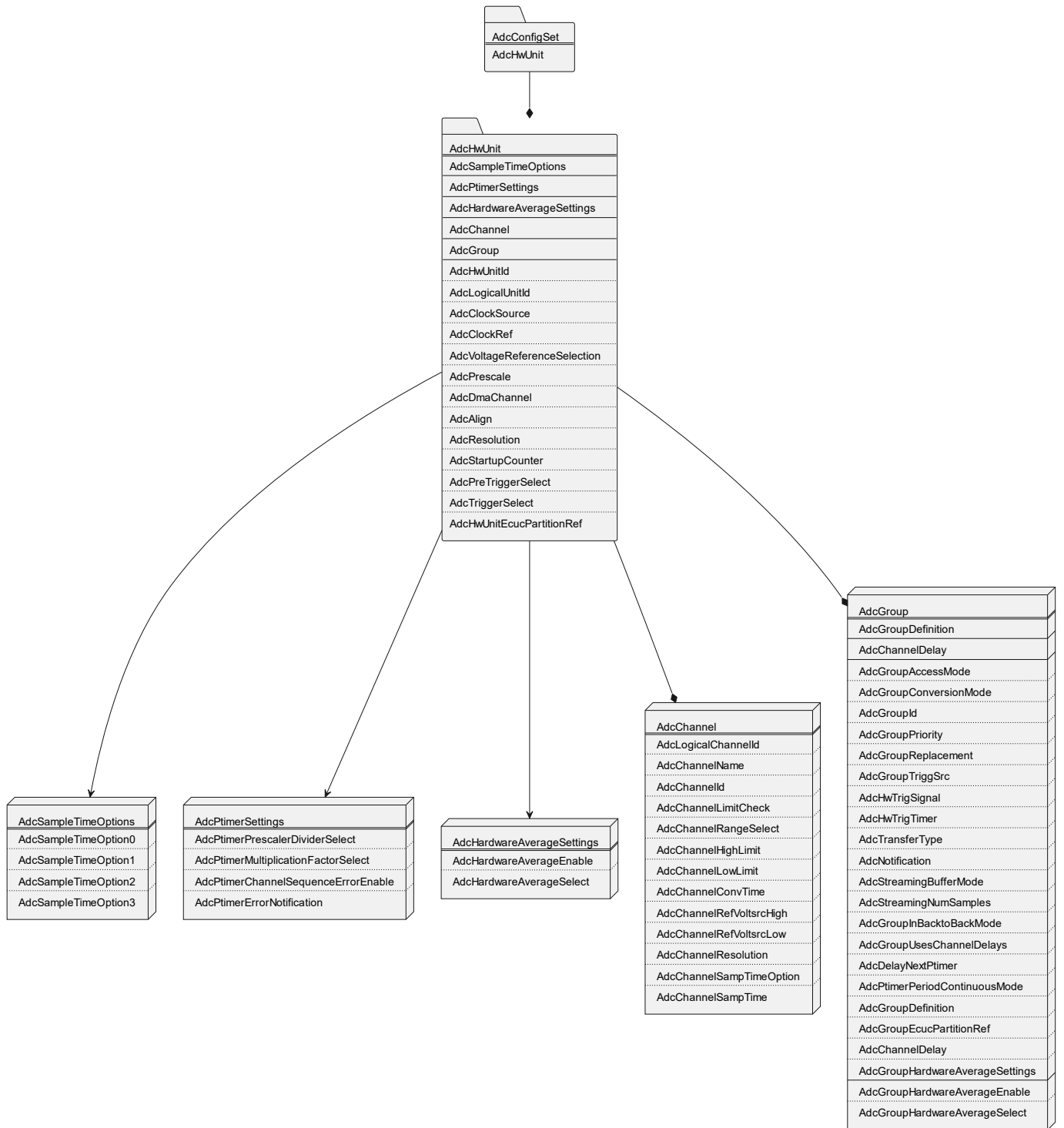
Adc
IMPLEMENTATION_CONFIG_VARIANT
AdcConfigSet
AdcGeneral
AdcInterrupt
AdcPublishedInformation
CommonPublishedInformation
AutosarExt
AdcDemEventParameterRefs

And the content of each container is as follows:

3.1.1 IMPLEMENTATION_CONFIG_VARIANT

IMPLEMENTATION_CONFIG_VARIANT

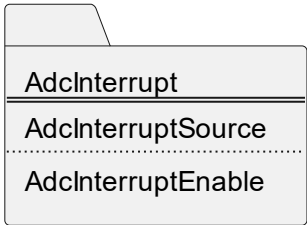
3.1.2 AdcConfigSet



3.1.3 AdcGeneral



3.1.4 AdcInterrupt



3.1.5 AdcPublishedInformation

AdcPublishedInformation
AdcChannelValueSigned
AdcGroupFirstChannelFixed
AdcMaxChannelResolution

CommonPublishedInformation
ArReleaseMajorVersion
ArReleaseMinorVersion
ArReleaseRevisionVersion
ModuleId
SwMajorVersion
SwMinorVersion
SwPatchVersion
VendorId

3.1.6 AutosarExt

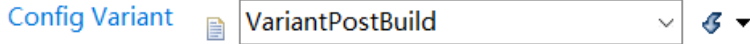
AutosarExt
AdcMulticoreSupport
AdcIsrPartitionMappingCheck
AdcDisableDemReportErrorStatus
AdcHardwareAverageOnce
AdcEnableDmaTrasferMode
AdcEnableUserModeSupport
AdcContinuousWithoutInterrupt

3.1.7 AdcDemEventParameterRefs

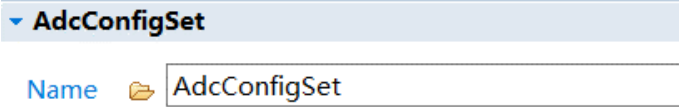
AdcDemEventParameterRefs
ADC_E_TIMEOUT

3.2 Containers and Variables

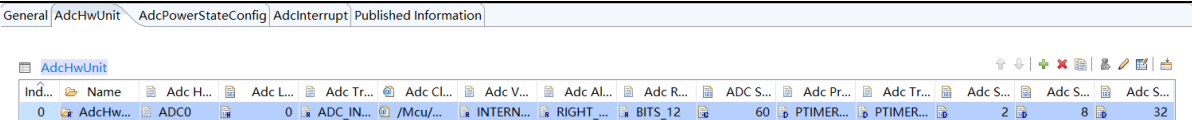
3.2.1 IMPLEMENTATION_CONFIG_VARIANT

Container	IMPLEMENTATION_CONFIG_VARIANT	
Description	N/A	
Screenshot		
Properties	Property	Value
	Type	Variable: enumeration
	Label	Config Variant
	Default	VariantPostBuild
	Range	VariantPostBuild VariantPreCompile
Returns	N/A	

3.2.2 AdcConfigSet

Container	AdcConfigSet	
Description	This container contains the configuration parameters and sub containers of the AUTOSAR ADC module.	
Screenshot		
Properties	Property	Container
	Type	Identifiable

3.2.2.1 AdcHwUnit

Container	AdcHwUnit	
Description	This container contains the Driver configuration (parameters) depending on grouping of channels.	
Screenshot		
Properties	Property	Container
	Type	Map
	Min	1

3.2.2.1.1 AdcHwUnitId

Container	AdcHwUnitId	
Description	Numeric ID of the HW Unit. This symbolic name allows accessing Hw Unit data.	
Screenshot		
Properties	Property	Value
	Type	Variable: Enumeration
	Range	ADC0, ADC1, ADC2, ADC3

3.2.2.1.2 AdcLogicalUnitId

Variable	AdcLogicalUnitId	
Description	Specifies the Logical id of the Hardware Unit.	
Screenshot		
Properties	Property	Value
	Type	Variable: Integer
	Range	<= 3, >=0

3.2.2.1.3 AdcClockSource

Variable	AdcClockSource	
Description	The ADC module specific clock input for the conversion unit can statically be configured to select different clock sources if provided by hardware. The hardware implementation does not support modification of clock source.	
Screenshot		
Properties	Property	Value
	Type	Variable: Enumeration

3.2.2.1.4 AdcClockRef

Variable	AdcClockRef	
Description	Reference to the ADC clock source configuration, which is set in the MCU driver configuration.	
Screenshot		
Properties	Property	Value
	Type	Variable: Reference
	Ref	Reference to node: McuClockReferencePoint

3.2.2.1.5 AdcVoltageReferenceSelection

Variable	AdcVoltageReferenceSelection	
Description	Selects the voltage reference source used for conversions. Refer to the chip configuration chapter's ADC section for details of these field definitions for this chip. • INTERNAL_REF - Selects internal voltage reference. • EXTERNAL_REF - Selects external voltage reference.	
Screenshot		
Properties	Property	Value
	Type	Variable: Enumeration
	Range	INTERNAL_REF, EXTERNAL_REF
	Default	INTERNAL_REF

3.2.2.1.6 AdcPrescale

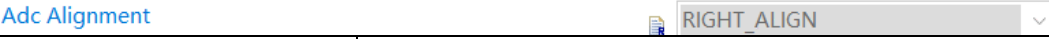
Variable	AdcPrescale	
Description	Optional ADC module specific clock prescale factor. • 1 - Clock not divided.	

	• 2 - Clock divided by 2. • 4 - Clock divided by 4. • 8 - Clock divided by 8.	
Screenshot		
Properties	Property	Value
	Type	Variable: Integer
	Range	1, 2, 4, 8

3.2.2.1.7 AdcDmaChannel

Variable	AdcDmaChannel	
Description	Dma channel associated to the Adc instance.	
Screenshot		
Properties	Property	Value
	Type	Variable: Reference
	Range	Reference to node: DmaChannelConfig

3.2.2.1.8 AdcAlign

Variable	AdcAlign	
Description	Used to set the ADC Data alignment. • LEFT_ALIGN - select left alignment. • RIGHT_ALIGN - select right alignment.	
Screenshot		
Properties	Property	Value
	Type	Variable: Enumeration
	Range	RIGHT_ALIGN, LEFT_ALIGN
	Default	RIGHT_ALIGN

3.2.2.1.9 AdcResolution

Variable	AdcResolution	
Description	Used to set the ADC resolution mode. • BITS_8 - 8-bit conversion. • BITS_10 - 10-bit conversion. • BITS_12 - 12-bit conversion.	
Screenshot		
Properties	Property	Value
	Type	Variable: Enumeration
	Default	BITS_12

3.2.2.1.10 AdcStartupCounter

Variable	AdcStartupCounter	
Description	The ADC needs a stabilization time before it starts converting accurately. This time is calculated by	

	counting QCLK cycles until the internal counter reaches this value. So user should set these bits before ADC enable.	
Screenshot		
Properties	Property	Value
	Type	Variable: Integer
	Range	<= 255, >=2
	Default	Calculated by ADC Clock Reference to make the counter time to be around 5 us.

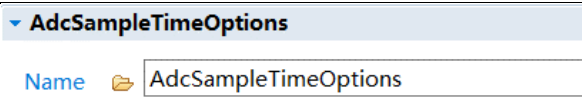
3.2.2.1.11 AdcPreTriggerSelect

Variable	AdcPreTriggerSelect	
Description	Selects the pre-trigger source of the ADC unit. Currently only Ptimer pre-trigger is supported. • 0 - Ptimer pre-trigger. • 1 - Software pre-trigger.	
Screenshot		
Properties	Property	Value
	Type	Variable: Enumeration
	Range	PTIMER_PRE_TRIGGER, SOFTWARE_PRE_TRIGGER
	Default	PTIMER_PRETRIGGER

3.2.2.1.12 AdcTriggerSelect

Variable	AdcTriggerSelect	
Description	Selects the trigger source of the ADC unit. Currently only Ptimer trigger is supported. • 0 - Ptimer trigger. • 1 - TrgSel trigger.	
Screenshot		
Properties	Property	Value
	Type	Variable: Enumeration
	Range	PTIMER_TRIGGER, TRGSEL_TRIGGER
	Default	PTIMER_TRIGGER

3.2.2.1.13 AdcSampleTimeOptions

Container	AdcSampleTimeOptions	
Description	Configuration for the ADC hardware sample time options.	
Screenshot		
Properties	Property	Container
	Type	Identifiable

3.2.2.1.13.1 AdcSampleTimeOption0

Variable	AdcSampleTimeOption0
-----------------	----------------------

Description	Configuration for the ADC hardware sample time option 0.	
Screenshot	Adc Sample Time Option 0 (2 -> 255) 	
Properties	Property	Value
	Type	Variable: Integer
	Range	<= 255, >=2
	Default	2

3.2.2.1.13.2 AdcSampleTimeOption1

Variable	AdcSampleTimeOption1	
Description	Configuration for the ADC hardware sample time option 1.	
Screenshot	Adc Sample Time Option 1 (2 -> 255) 	
Properties	Property	Value
	Type	Variable: Integer
	Range	<= 255, >=2
	Default	8

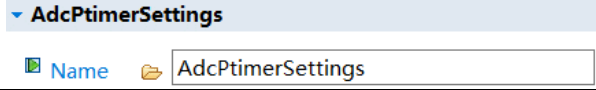
3.2.2.1.13.3 AdcSampleTimeOption2

Variable	AdcSampleTimeOption2	
Description	Configuration for the ADC hardware sample time option 2.	
Screenshot	Adc Sample Time Option 2 (2 -> 255) 	
Properties	Property	Value
	Type	Variable: Integer
	Range	<= 255, >=2
	Default	32

3.2.2.1.13.4 AdcSampleTimeOption3

Variable	AdcSampleTimeOption3	
Description	Configuration for the ADC hardware sample time option 3.	
Screenshot	Adc Sample Time Option 3 (2 -> 255) 	
Properties	Property	Value
	Type	Variable: Integer
	Range	<= 255, >=2
	Default	128

3.2.2.1.14 AdcPtimerSettings

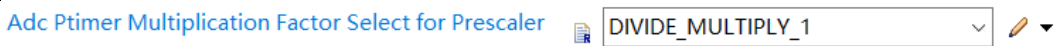
Container	AdcPtimerSettings	
Description	Configuration for the Ptimer unit associated with this ADC hardware unit.	
Screenshot		
Properties	Property	Container

	Type	Identifiable
--	------	--------------

3.2.2.1.14.1 AdcPtimerPrescalerDividerSelect

Variable	AdcPtimerPrescalerDividerSelect	
Description	Determines the prescaler used for the Ptimer counter. - DIVIDE_BY_1 - Counting uses the peripheral clock divided by multiplication factor selected by AdcPtimerMultiplicationFactorSelect. - DIVIDE_BY_2 - Counting uses the peripheral clock divided by twice of the multiplication factor selected by AdcPtimerMultiplicationFactorSelect. - DIVIDE_BY_4 - Counting uses the peripheral clock divided by four of the multiplication factor selected by AdcPtimerMultiplicationFactorSelect. - DIVIDE_BY_8 - Counting uses the peripheral clock divided by eight of the multiplication factor selected by AdcPtimerMultiplicationFactorSelect. - DIVIDE_BY_16 - Counting uses the peripheral clock divided by 16 times of the multiplication factor selected by AdcPtimerMultiplicationFactorSelect. - DIVIDE_BY_32 - Counting uses the peripheral clock divided by 32 times of the multiplication factor selected by AdcPtimerMultiplicationFactorSelect. - DIVIDE_BY_64 - Counting uses the peripheral clock divided by 64 times of the multiplication factor selected by AdcPtimerMultiplicationFactorSelect. - DIVIDE_BY_128 - Counting uses the peripheral clock divided by 128 times of the multiplication factor selected by AdcPtimerMultiplicationFactorSelect.	
Screenshot		
Properties	Property	Value
	Type	Variable: Enumeration
	Range	DIVIDE_BY_1, DIVIDE_BY_2, DIVIDE_BY_4, DIVIDE_BY_8, DIVIDE_BY_16, DIVIDE_BY_32, DIVIDE_BY_64, DIVIDE_BY_128
	Default	DIVIDE_BY_1

3.2.2.1.14.2 AdcPtimerMultiplicationFactorSelect

Variable	AdcPtimerMultiplicationFactorSelect	
Description	Selects the multiplication factor of the prescaler divider for the counter clock of the associated Ptimer unit. - DIVIDE_MULTIPLY_1 - Multiplication factor is 1. - DIVIDE_MULTIPLY_10 - Multiplication factor is 10. - DIVIDE_MULTIPLY_20 - Multiplication factor is 20. - DIVIDE_MULTIPLY_40 - Multiplication factor is 40.	
Screenshot		
Properties	Property	Value
	Type	Variable: Enumeration
	Range	DIVIDE_MULTIPLY_1, DIVIDE_MULTIPLY_10, DIVIDE_MULTIPLY_20, DIVIDE_MULTIPLY_40
	Default	DIVIDE_MULTIPLY_1

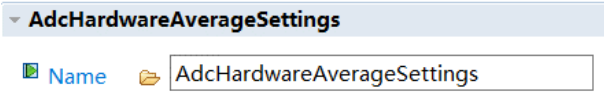
3.2.2.1.14.3 AdcPtimerChannelSequenceErrorEnable

Variable	AdcPtimerChannelSequenceErrorEnable	
Description	Enables the Ptimer sequence error interrupt. When this field is set, any of the Ptimer channel sequence error flags generates a Ptimer sequence error interrupt.	
Screenshot	    	
Properties	Property	Value
	Type	Variable: Boolean
	Default	False

3.2.2.1.14.4 AdcPtimerErrorNotification

Variable	AdcPtimerErrorNotification	
Description	Callback function for Ptimer channel sequence error. This function pointer is called every time when Ptimer channel sequence error occurred.	
Screenshot	  NULL_PTR  	
Properties	Property	Value
	Type	Variable: FunctionName
	Default	NULL_PTR

3.2.2.1.15 AdcHardwareAverageSettings

Container	AdcHardwareAverageSettings	
Description	Selects hardware average samples used at initialization.	
Screenshot		
Properties	Property	Container
	Type	Identifiable

3.2.2.1.15.1 AdcHardwareAverageEnable

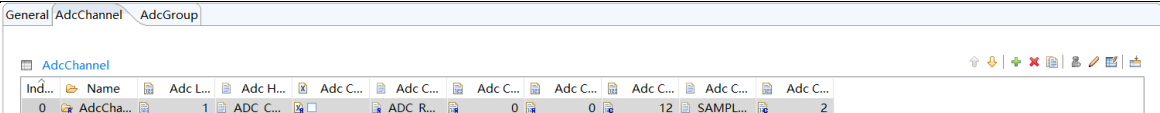
Variable	AdcHardwareAverageEnable	
Description	Enables the hardware average function of the ADC.	
Screenshot	    	
Properties	Property	Value
	Type	Variable: Boolean
	Default	False

3.2.2.1.15.2 AdcHardwareAverageSelect

Variable	AdcHardwareAverageSelect	
Description	Determines how many ADC conversions will be averaged to create the ADC average result. • SAMPLES_4 - 4 samples averaged. • SAMPLES_8 - 8 samples averaged. • SAMPLES_16 - 16 samples averaged. • SAMPLES_32 - 32 samples averaged.	

Screenshot		
Properties	Property	Value
	Type	Variable: Enumeration
	Range	SAMPLES_4, SAMPLES_8, SAMPLES_16, SAMPLES_32
	Default	SAMPLES_4

3.2.2.1.16 AdcChannel

Container	AdcChannel	
Description	This container contains the channel configuration (parameters) depending on the hardware capability.	
Screenshot		
Properties	Property	Container
	Type	Map
	Min	1

3.2.2.1.16.1 AdcLogicalChannelId

Variable	AdcLogicalChannelId	
Description	This is the logical Id of the ADC channel.	
Screenshot		
Properties	Property	Value
	Type	Variable: Integer
	Range	<=1024, >=0
	Default	0

3.2.2.1.16.2 AdcChannelName

Variable	AdcChannelName	
Description	This parameter defines the assignment of the channel to the physical ADC hardware channel. Note: Range of the ADC Channels depends on the selected package.	
Screenshot		
Properties	Property	Value
	Type	Variable: Enumeration
	Range	Reference to the property file of the MCU
	Default	ADC0_SE0

3.2.2.1.16.3 AdcChannelId

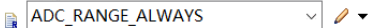
Variable	AdcChannelId	
Description	This parameter defines the assignment of the channel to the physical ADC hardware channel. Note: This field is in sync with AdcChannelName, you should always use the calculate button to calculate the appropriate value.	
Screenshot		

Properties	Property	Value
	Type	Variable: Integer
	Range	Reference to the property file of the MCU
	Default	0

3.2.2.1.16.4 AdcChannelLimitCheck

Variable	AdcChannelLimitCheck	
Description	Enables or disables limit checking for an ADC channel.	
Screenshot	Adc Channel Limit Check 	
Properties	Property	Value
	Type	Variable: Boolean
	Default	False

3.2.2.1.16.5 AdcChannelRangeSelect

Variable	AdcChannelRangeSelect	
Description	In case of active limit checking: defines which conversion values are taken into account related to the borders defined with AdcChannelLowLimit and AdcChannelHighLimit.	
Screenshot	Adc Channel Range Select 	
Properties	Property	Value
	Type	Variable: Enumeration
	Range	ADC_RANGE_ALWAYS, ADC_RANGE_BETWEEN, ADC_RANGE_NOT_BETWEEN, ADC_RANGE_NOT_OVER_HIGH, ADC_RANGE_NOT_UNDER_LOW, ADC_RANGE_OVER_HIGH, ADC_RANGE_UNDER_LOW
	Default	ADC_RANGE_ALWAYS

3.2.2.1.16.6 AdcChannelHighLimit

Variable	AdcChannelHighLimit	
Description	High limit - used for limit checking.	
Screenshot	Adc Channel High Limit 	
Properties	Property	Value
	Type	Variable: Integer
	Range	<=65535, >=0
	Default	0

3.2.2.1.16.7 AdcChannelLowLimit

Variable	AdcChannelLowLimit	
Description	Low limit - used for limit checking.	
Screenshot	Adc Channel Low Limit 	
Properties	Property	Value

	Type	Variable: Integer
	Range	<=65535, >=0
	Default	0

3.2.2.1.16.8 AdcChannelConvTime

Variable	AdcChannelConvTime	
Description	Configuration of conversion time, i.e. the time during which the analogue value is converted into digital representation, (in clock cycles) for each channel, if supported by hardware. This parameter is not used by the current implementation.	
Screenshot		
Properties	Property	Value
	Type	Variable: Integer
	Range	<= 18446744073709551615, >=0
	Default	0

3.2.2.1.16.9 AdcChannelRefVoltsrcHigh

Variable	AdcChannelRefVoltsrcHigh	
Description	Upper reference voltage source for each channel. This parameter is not used by the current implementation.	
Screenshot		
Properties	Property	Value
	Type	Variable: Enumeration
	Range	UPPER_REF_VOLT_0
	Default	UPPER_REF_VOLT_0

3.2.2.1.16.10 AdcChannelRefVoltsrcLow

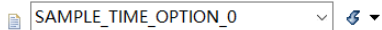
Variable	AdcChannelRefVoltsrcLow	
Description	Lower reference voltage source for each channel. This parameter is not used by the current implementation.	
Screenshot		
Properties	Property	Value
	Type	Variable: Enumeration
	Range	LOWER_REF_VOLT_0
	Default	LOWER_REF_VOLT_0

3.2.2.1.16.11 AdcChannelResolution

Variable	AdcChannelResolution	
Description	Channel Resolution in bits of converted value.	
Screenshot		
Properties	Property	Value

	Type	Variable: Integer
	Range	Reference to AdcResolution field in AdcHwUnit Config
	Default	Reference to AdcResolution field in AdcHwUnit Config

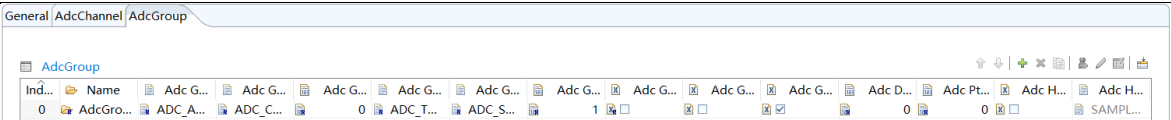
3.2.2.1.16.12 AdcChannelSampTimeOption

Variable	AdcChannelSampTimeOption	
Description	Configuration of sampling time option, the sample times are configured in the AdcHwUnit. This determines the sample time of the channel, i.e. the time during which the value is sampled, (in clock cycles) for each channel.	
Screenshot		
Properties	Property	Value
	Type	Variable: Enumeration
	Range	SAMPLE_TIME_OPTION_0, SAMPLE_TIME_OPTION_1, SAMPLE_TIME_OPTION_2, SAMPLE_TIME_OPTION_3
	Default	SAMPLE_TIME_OPTION_0

3.2.2.1.16.13 AdcChannelSampTime

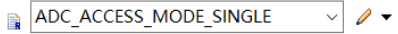
Variable	AdcChannelSampTime	
Description	Configuration of sampling time, i.e. the time during which the value is sampled, (in clock cycles) for each channel, if supported by hardware. This parameter shall be updated by select ADC Channel Sample Time Option.	
Screenshot		
Properties	Property	Value
	Type	Variable: Integer
	Range	This value is determined by the AdcChannelSampTimeOption and AdcSampleTimeOptions in AdcHwUnit
	Default	This value is determined by the AdcChannelSampTimeOption and AdcSampleTimeOptions in AdcHwUnit

3.2.2.1.17 AdcGroup

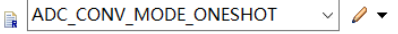
Container	AdcGroup	
Description	This container contains the Group configuration (parameters).	
Screenshot		
Properties	Property	Container
	Type	Map
	Min	1

3.2.2.1.17.1 AdcGroupAccessMode

Variable	AdcGroupAccessMode
-----------------	--------------------

Description	Type of access mode to group conversion results.	
Screenshot	Adc Group Access Mode 	
Properties	Property	Value
	Type	Variable: Enumeration
	Range	ADC_ACCESS_MODE_SINGLE, ADC_ACCESS_MODE_STREAMING
	Default	ADC_ACCESS_MODE_SINGLE
Requirements	ECUC_Adc_00317	

3.2.2.1.17.2 AdcGroupConversionMode

Variable	AdcGroupConversionMode	
Description	Type of Conversion mode of the channel group.	
Screenshot	Adc Group Conversion Mode 	
Properties	Property	Value
	Type	Variable: Enumeration
	Range	ADC_CONV_MODE_CONTINUOUS, ADC_CONV_MODE_ONESHOT
	Default	ADC_CONV_MODE_CONTINUOUS

3.2.2.1.17.3 AdcGroupId

Variable	AdcGroupId	
Description	Numeric ID of the group. This parameter is the symbolic name to be used on the API. This symbolic name allows accessing Channel Group data. This value will be assigned to the symbolic name derived of the AdcGroup container shortName.	
Screenshot	Adc Group Id 	
Properties	Property	Value
	Type	Variable: Integer
	Range	<=1023, >=0
	Default	0

3.2.2.1.17.4 AdcGroupPriority

Variable	AdcGroupPriority	
Description	Priority level of the AdcGroup. This item is ignored if Adc/AdcGeneral/AdcPriorityImplementation is defined to ADC_PRIORITY_NONE.	
Screenshot	 Adc Group Priority 	
Properties	Property	Value
	Type	Variable: Integer
	Range	<=255, >=0
	Default	0

3.2.2.1.17.5 AdcGroupReplacement

Variable	AdcGroupReplacement	
-----------------	---------------------	--

Description	Replacement mechanism used on ADC group level, if a group conversion is interrupted by a group which has a higher priority. It's fixed to Abort/Restart	
Screenshot		
Properties	Property	Value
	Type	Variable: Enumeration
	Range	ADC_GROUP_REPL_ABORT_RESTART, ADC_GROUP_REPL_SUSPEND_RESUME
	Default	ADC_GROUP_REPL_ABORT_RESTART

3.2.2.1.17.6 AdcGroupTriggSrc

Variable	AdcGroupTriggSrc	
Description	Type of source event that starts a group conversion. It's possible to select HW or SW trigger. In case of HW trigger, the trigger source can be from the Ptimer. Only trigger sources from Ptimer are used by the current implementation. The trigger source connected to Ptimer can be configured from TrgSel module.	
Screenshot		
Properties	Property	Value
	Type	Variable: Enumeration
	Range	ADC_TRIGG_SRC_HW, ADC_TRIGG_SRC_SW
	Default	ADC_TRIGG_SRC_SW

3.2.2.1.17.7 AdcHwTrigSignal

Variable	AdcHwTrigSignal	
Description	Configures on which edge of the hardware trigger signal the driver should react, i.e. start the conversion. Note: Only ADC_HW_TRIG_RISING_EDGE is used by the current implementation.	
Screenshot		
Properties	Property	Value
	Type	Variable: Enumeration
	Range	ADC_HW_TRIG_BOTH_EDGES, ADC_HW_TRIG_FALLING_EDGE, ADC_HW_TRIG_RISING_EDGE
	Default	ADC_HW_TRIG_RISING_EDGE

3.2.2.1.17.8 AdcHwTrigTimer

Variable	AdcHwTrigTimer	
Description	Reload value of the ADC module embedded timer. This parameter is not used by the current implementation	
Screenshot		
Properties	Property	Value
	Type	Variable: Integer
	Range	<= 18446744073709551615, >=0
	Default	0

3.2.2.1.17.9 **AdcTransferType**

Variable	AdcTransferType	
Description	Select the transfer Type. If DMA is used, user must not run SW and HW groups at the same time on the same HW unit because the same DMA channel will be used for both.	
Screenshot		
Properties	Property	Value
	Type	Variable: Enumeration
	Range	ADC_DMA, ADC_INTERRUPT, ADC_POLLING
	Default	ADC_INTERRUPT

3.2.2.1.17.10 **AdcNotification**

Variable	AdcNotification	
Description	Callback function for each group. This function pointer is called whenever the conversion of this group is completed.	
Screenshot		
Properties	Property	Value
	Type	Variable: FuctionName
	Default	NULL_PTR

3.2.2.1.17.11 **AdcStreamingBufferMode**

Variable	AdcStreamingBufferMode	
Description	Configure streaming buffer as linear buffer (i.e. the ADC Driver stops the conversion as soon as the stream buffer is full) or as circular buffer (wraps around if the end of the stream buffer is reached).	
Screenshot		
Properties	Property	Value
	Type	Variable: Enumeration
	Range	ADC_STREAM_BUFFER_CIRCULAR, ADC_STREAM_BUFFER_LINEAR
	Default	ADC_STREAM_BUFFER_LINEAR

3.2.2.1.17.12 **AdcStreamingNumSamples**

Variable	AdcStreamingNumSamples	
Description	Number of ADC values to be acquired per channel in streaming access mode. Note: In single access mode this parameter assumes value 1 since only one sample per channel is processed.	
Screenshot		
Properties	Property	Value
	Type	Variable: Integer
	Range	<=1024, >=1
	Default	1

3.2.2.1.17.13 **AdcGroupInBacktoBackMode**

Variable	AdcGroupInBacktoBackMode
-----------------	--------------------------

Description	Enable/ Disable the channel conversions occurring in Back to Back mode. The Ptimer hardware unit is used to initiate individual channel conversions that are required for each group conversion, with one Ptimer pretrigger controlling the timing of each ADC channel conversion. When <code>AdcGroupInBacktoBackMode</code> is enabled, the Ptimer pretriggers will be configured to work in back to back mode: <code>PTIMER_CTRL_CHn[BTB]</code> bits are set for all the group channels, except the first one. The first channel conversion complete will trigger the Ptimer channel's second pretrigger, and so on until the last channel of the group. This ensures that the channel conversions occur in sequence, as quickly as possible and without causing internal Ptimer errors. The Adc hardware has only 32 channels per instance. For groups larger than 32 channels, Adc driver will split the group in sub-groups of 32 channels and convert them with sub-group conversions. Interrupt processing will be involved for register re-configuration. The Ptimer hardware has 4 channels and each channel could trigger 8 Adc channels. Within one Ptimer channel, pretriggers could work in back to back mode. However, back to back mode cannot work between Ptimer channels. So a delay is needed for the channels after the first Ptimer channel, otherwise Ptimer errors will occur. Adc driver will configure this delay value when Back to back mode is used, with the value defined by the user in <code>AdcDelayNextPtimer</code> parameter, multiplied by the channel number. The sequences of 8 channels associated to a Ptimer channel will happen in back to back mode. If <code>AdcGroupInBacktoBackMode</code> is not enabled for a group, all pretriggers of the Ptimer channels will be triggered at the same time causing internal Ptimer errors, unless the coherence of channel conversions is ensured via the delay configured for each pretrigger. This is why at least one of <code>AdcGroupInBacktoBackMode</code> and <code>AdcGroupUsesChannelDelays</code> options needs to be configured for an Adc Group. If both parameters are configured for the Adc Group, the user can configure a delay only for the first channel conversion, and the rest will be converted in back to back mode. If <code>AdcGroupUsesChannelDelays</code> is not configured for the group, the first channel will have an associated delay of 0, and will convert immediately after the external trigger is received.	
Screenshot	Adc Group In Back to Back Mode   	
Properties	Property	Value
	Type	Variable: Boolean
	Default	False

3.2.2.1.17.14 `AdcGroupUsesChannelDelays`

Variable	<code>AdcGroupUsesChannelDelays</code>
Description	Enable/ Disable the usage of channel delays in Ptimer pretriggers. The Ptimer hardware unit is used to initiate individual channel conversions that are required for each group conversion, with one Ptimer pretrigger controlling the timing of each ADC channel conversion. When <code>AdcGroupUsesChannelDelays</code> is enabled, the Ptimer pretriggers will have individual delays configured. <code>PTIMER_DLYx_CHn</code> will be set with the values configured in <code>AdcChannelDelay</code> list for each channel. The values must not be equal or too close, otherwise Ptimer internal errors will occur. The Adc driver can convert only up to 32 channels at once; for groups larger than 32 channels, Adc driver will split the group in sub-groups of 32 and convert them with sub-group conversions. Interrupt processing is needed between the sub-groups for register re-configuration. The timing of the channel conversions will be accurate only for the first 32 channels; for the next ones, the timing of the interrupt processing will be implicitly added. If <code>AdcGroupInBacktoBackMode</code> is also configured for the Adc Group, the user can configure a delay only for the first channel conversion, and the rest will be converted in back to back mode (see description of <code>AdcGroupInBacktoBackMode</code>). The delay value defined by the user in <code>AdcDelayNextPtimer</code> parameter will be used for following Ptimer

	channels to avoid Ptimer internal errors.	
Screenshot	Adc Group Uses Channel Delays    ▼	
Properties	Property	Value
	Type	Variable: Boolean
	Default	False

3.2.2.1.17.15 AdcDelayNextPtimer

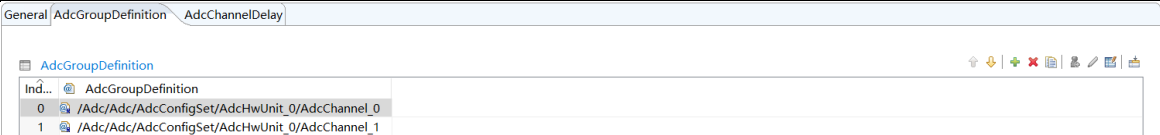
Variable	AdcDelayNextPtimer	
Description	The Ptimer unit can have one or more Ptimer Channels of 8 pretriggers each that will be used together for setting up conversions of large groups. The Ptimer Channels will be started together when the Ptimer unit is triggered, thus risking to request conversions at the same time and causing internal errors. For this reason, the value of AdcDelayNextPtimer is needed to be added as delay between Ptimer channels when Back to Back mode is used. The AdcDelayNextPtimer value should be greater than the timing to convert 8 channels. The maximum total conversion time is determined by the clock source chosen and the divide ratio selected. The clock source is configured in MCU, and the divide ratio is specified by ADC_CFG2[DIV] (not available in FC4150F512K). To calculate total conversion time for one channel the following formula is applied: ADC TOTAL CONVERSION TIME = Sample Phase Time (set by ADC_SMPR[SMP_OPTn]+1) + Hold Phase (1 ADC Cycle) + Compare Phase Time (8-bit Mode=20 ADC Cycles, 10-bit Mode=24 ADC Cycles, 12-bit Mode=28 ADC Cycles) + Single or First continuous time adder (5 ADC cycles + 5 bus clock cycles).	
Screenshot	Adc Delay Next PTIMER (0 -> 65535)  0  ▼	
Properties	Property	Value
	Type	Variable: Integer
	Range	<=65535, >=1
	Default	0

3.2.2.1.17.16 AdcPtimerPeriodContinuousMode

Variable	AdcPtimerPeriodContinuousMode	
Description	The period of Ptimer module in continuous mode. The Ptimer period should be big enough to ensure all of the channels complete the conversion before Ptimer restarts.	
Screenshot	Adc Ptimer Period For Continous Mode (0 -> 65535)  8000  ▼	
Properties	Property	Value
	Type	Variable: Integer
	Range	<=65535, >=1
	Default	0

3.2.2.1.17.17 AdcGroupDefinition

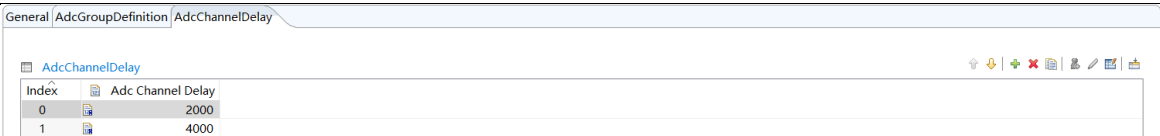
List	AdcGroupDefinition
Description	Assignment of AdcChannels to a AdcGroups. For each AdcChannel that should belong to the group, a reference needs to be defined.

Screenshot		
Properties	Property	List
	Type	Variable: Reference
	Range	Reference to AdcChannel
	Min	1

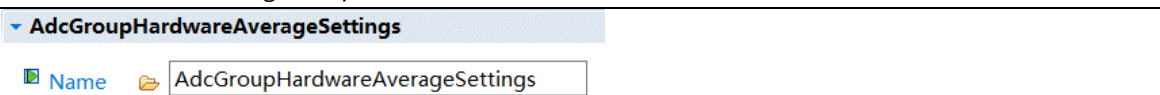
3.2.2.1.17.18 AdcGroupEcucPartitionRef

List	AdcGroupEcucPartitionRef	
Description	Maps an ADC channel group to zero or multiple ECUC partitions to limit the access to this channel group. The ECUC partitions referenced are a subset of the ECUC partitions where the ADC driver is mapped to. Since the driver implementation only supports multi-core type II, this option is not supported.	
Screenshot		
Properties	Property	List
	Type	Variable: Reference
	Range	Reference to AdcEcucPartitionRef
	Min	0

3.2.2.1.17.19 AdcChannelDelay

List	AdcChannelDelay	
Description	Delay of the associated Ptimer pretrigger for this channel. If Back to Back mode is not used for this group, the user must ensure the difference between the delays of different channels are big enough to accommodate the ADC conversions without Ptimer channel sequence errors. If Back to Back mode is used for this group, only the delay for the first channel will be considered - the rest of the channels will be converted in back to back mode. The delay value has to be a increasing number for each sequence of 32.	
Screenshot		
Properties	Property	List
	Type	Variable: Integer
	Range	<=65535, >=0
	Default	Index * 50

3.2.2.1.17.20 AdcGroupHardwareAverageSettings

Container	AdcGroupHardwareAverageSettings	
Description	Selects hardware average samples used at initialization.	
Screenshot		
Properties	Property	Container

	Type	Identifiable
--	------	--------------

3.2.2.1.17.20.1 *AdcGroupHardwareAverageEnable*

Variable	AdcGroupHardwareAverageEnable	
Description	Enables the hardware average function of the ADC.	
Screenshot	Adc Hardware Average Enable    	
Properties	Property	Value
	Type	Variable: Boolean
	Default	False

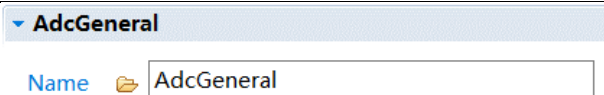
3.2.2.1.17.20.2 *AdcGroupHardwareAverageSelect*

Variable	AdcGroupHardwareAverageSelect	
Description	Determines how many ADC conversions will be averaged to create the ADC average result. This functionality is activated when ADC_CFG2[AVG_EN] = 1. • SAMPLES_4 - 4 samples averaged. • SAMPLES_8 - 8 samples averaged. • SAMPLES_16 - 16 samples averaged. • SAMPLES_32 - 32 samples averaged.	
Screenshot	Adc Hardware Average Enable    	
Properties	Property	Value
	Type	Variable: Enumeration
	Range	SAMPLES_4, SAMPLES_8, SAMPLES_16, SAMPLES_32
	Default	SAMPLES_4

3.2.2.1.18 *AdcHwUnitEcucPartitionRef*

List	AdcHwUnitEcucPartitionRef	
Description	Maps a ADC hardware unit to zero or one ECUC partition to limit the access to this hardware unit. The ECUC partitions referenced are a subset of the ECUC partitions where the ADC driver is mapped to. This node is EDITABLE only if "Adc Multicore Support" is enabled.	
Screenshot		
Properties	Property	List
	Type	Variable: Reference
	Range	Reference to AdcEcucPartitionRef
	Min	0

3.2.3 *AdcGeneral*

Container	AdcGeneral	
Description	General configuration (parameters) of the ADC Driver software module.	
Screenshot		

Properties	Property	Container
	Type	Identifiable

3.2.3.1 AdcDelInitApi

Variable	AdcDelInitApi	
Description	Adds/removes the service Adc_Delnit() from the code. - true: Adc_Delnit() can be used. - false: Adc_Delnit() can not be used.	
Screenshot	Adc_Delnit API   	
Properties	Property	Value
	Type	Variable: Boolean
	Default	True

3.2.3.2 AdcDevErrorDetect

Variable	AdcDevErrorDetect	
Description	Switches the Default Error Tracer (Det) detection and notification ON or OFF.true: Enabled (ON). false: Disabled (OFF).	
Screenshot	Adc Development Error Detection   	
Properties	Property	Value
	Type	Variable: Boolean
	Default	False

3.2.3.3 AdcEnableLimitCheck

Variable	AdcEnableLimitCheck	
Description	Enable/disable limit checking feature in the ADC driver.	
Screenshot	Adc Enable Limit Check   	
Properties	Property	Value
	Type	Variable: Boolean
	Default	False

3.2.3.4 AdcEnableQueuing

Variable	AdcEnableQueuing	
Description	Determines, if the queuing mechanism is active in case of priority mechanism disabled. Enable/Disable the Queue.Note that if AdcPriorityImplementation = ADC_PRIORITY_HW_SW, this field is always enabled.	
Screenshot	Adc Enable Queuing   	
Properties	Property	Value
	Type	Variable: Boolean
	Default	True

3.2.3.5 AdcEnableStartStopGroupApi

Variable	AdcEnableStartStopGroupApi
-----------------	----------------------------

Description	Adds / removes the services <code>Adc_StartGroupConversion()</code> and <code>Adc_StopGroupConversion()</code> from the code. true: <code>Adc_StartGroupConversion()</code> and <code>Adc_StopGroupConversion()</code> can be used. false: <code>Adc_StartGroupConversion()</code> and <code>Adc_StopGroupConversion()</code> can not be used.	
Screenshot	Adc_StartStopGroup API   	
Properties	Property	Value
	Type	Variable: Boolean
	Default	True

3.2.3.6 AdcGrpNotifCapability

Variable	AdcGrpNotifCapability	
Description	Determines, if the group notification mechanism (the functions to enable and disable the notifications) is available at runtime. - true: Enabled. - false: Disabled.	
Screenshot	Adc Group Notification Capability   	
Properties	Property	Value
	Type	Variable: Boolean
	Default	True

3.2.3.7 AdcHwTriggerApi

Variable	AdcHwTriggerApi	
Description	Adds / removes the services <code>Adc_EnableHardwareTrigger()</code> and <code>Adc_DisableHardwareTrigger()</code> from the code. - true: <code>Adc_EnableHardwareTrigger()</code> and <code>Adc_DisableHardwareTrigger()</code> can be used. - false: <code>Adc_EnableHardwareTrigger()</code> and <code>Adc_DisableHardwareTrigger()</code> can not be used.	
Screenshot	Adc Hw Trigger API   	
Properties	Property	Value
	Type	Variable: Boolean
	Default	False

3.2.3.8 AdcReadGroupApi

Variable	AdcReadGroupApi	
Description	Adds / removes the service <code>Adc_ReadGroup()</code> from the code.	
Screenshot	Adc_ReadGroup API   	
Properties	Property	Value
	Type	Variable: Boolean
	Default	True

3.2.3.9 AdcVersionInfoApi

Variable	AdcVersionInfoApi	
Description	Adds / removes the service <code>Adc_GetVersionInfo()</code> from the code. true: <code>Adc_GetVersionInfo()</code> can be used.	

	false: Adc_GetVersionInfo() can not be used.	
Screenshot	Adc_VersionInfo API   	
Properties	Property	Value
	Type	Variable: Boolean
	Default	True

3.2.3.10 AdcPriorityImplementation

Variable	AdcPriorityImplementation	
Description	Determines whether a priority mechanism is available for prioritization of the conversion requests and if available, the type of prioritization mechanism. The selection applies for groups with trigger source software and trigger source hardware. Two types of prioritization mechanism can be selected. The hardware prioritization mechanism (AdcPriorityHw) uses the ADC hardware features for prioritization of the software conversion requests and hardware trigger signals for groups with trigger source hardware. The mixed hardware and software prioritization mechanism (AdcPriorityHwSw) uses the ADC hardware features for prioritization of ADC hardware trigger for groups with trigger source hardware and a software implemented prioritization mechanism for groups with trigger source software. The group priorities for software triggered groups are typically configured with lower priority levels than the group priorities for hardware triggered groups. For this platform, the ADC_PRIORITY_HW isn't used.	
Screenshot	Adc Priority Mechanism  ADC_PRIORITY_NONE 	
Properties	Property	Value
	Type	Variable: Enumeration
	Range	ADC_PRIORITY_HW, ADC_PRIORITY_HW_SW, ADC_PRIORITY_NONE
	Default	ADC_PRIORITY_NONE

3.2.3.11 AdcResultAlignment

Variable	AdcResultAlignment	
Description	Alignment of ADC raw results in ADC result buffer (left/right alignment).	
Screenshot	Adc Result Alignment  ADC_ALIGN_RIGHT 	
Properties	Property	Value
	Type	Variable: Enumeration
	Range	ADC_ALIGN_LEFT, ADC_ALIGN_RIGHT
	Default	ADC_ALIGN_RIGHT

3.2.3.12 AdcTimeout

Variable	AdcTimeout	
Description	This is a timeout value which is used to wait till- ADC hardware is disabled- Conversion completed in calibration feature. If the Status is not updated then after this timeout the ADC_E_TIMEOUT production error will be reported and the rest of the functionality will be skipped	
Screenshot	Adc Timeout: (1000 -> 4294967295)  65535 	
Properties	Property	Value
	Type	Variable: Integer
	Range	<= 4294967295, >=1000

	Default	65535
--	---------	-------

3.2.3.13 AdcDmaTimeout

Variable	AdcDmaTimeout	
Description	This is a timeout value which is used to wait till Dma transferring finish. It's used to ensure the re-configure dma setting when Ptimer error occurs. If the Status is not updated, then after this timeout the ADC_E_TIMEOUT production error will be reported and the rest of the functionality will be skipped.	
Screenshot	 Adc Dma Transferring Timeout: (1000 -> 4294967295) 65535	
Properties	Property	Value
	Type	Variable: Integer
	Range	<= 4294967295, >=1000
	Default	65535

3.2.3.14 AdcPriorityQueueMaxDepth

Variable	AdcPriorityQueueMaxDepth	
Description	Maximum depth of queue used for queuing of incoming conversion requests when hardware unit is busy.	
Screenshot	 Adc Max Queue Depth 1	
Properties	Property	Value
	Type	Variable: Integer
	Range	<= 1024, >=1
	Default	1

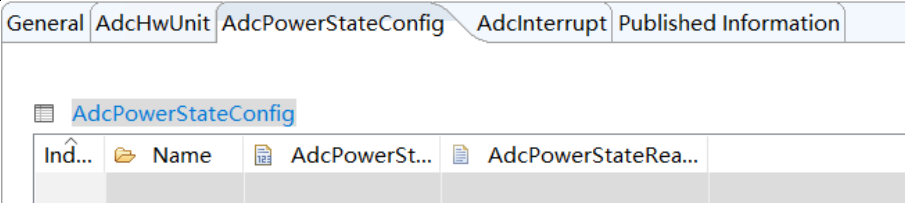
3.2.3.15 AdcLowPowerStatesSupport

Variable	AdcLowPowerStatesSupport	
Description	Adds / removes all power state management related APIs (Adc_SetPowerState, Adc_GetCurrentPowerState, Adc_GetTargetPowerState, Adc_PreparePowerState, Adc_Main_PowerTransitionManager), indicating if the HW offers low power state management. This parameter is disabled, there is no power management support implemented for this platform.	
Screenshot	 Adc Low Power States Support ☐	
Properties	Property	Value
	Type	Variable: Boolean
	Default	False

3.2.3.16 AdcPowerStateAsynchTransitionMode

Variable	AdcPowerStateAsynchTransitionMode	
Description	Enables / disables support of the ADCDriver to the asynchronous power state transition. This feature is not implemented on this platform.	
Screenshot	 Adc Power State Asynch Transition Mode ☐	
Properties	Property	Value
	Type	Variable: Boolean
	Default	False

3.2.3.17 AdcPowerStateConfig

Container	AdcPowerStateConfig	
Description	Each instance of this parameter defines a power state and the callback to be called when this power state is reached.	
Screenshot		
Properties	Property	List
	Type	Map

3.2.3.17.1 AdcPowerState

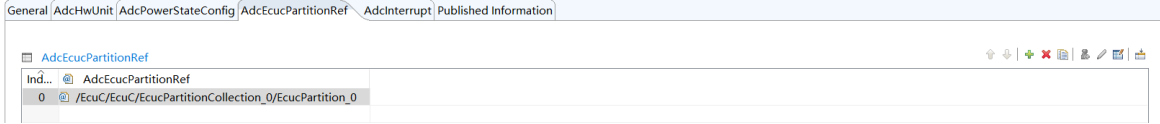
Variable	AdcPowerState	
Description	Each instance of this parameter describes a different power state supported by the ADC HW. It should be defined by the HW supplier and used by the ADCDriver to reference specific HW configurations which set the ADC HW module in the referenced power state. At least the power mode corresponding to full power state shall be always configured. This parameter shall only be configured if the parameter AdcLowPowerStatesSupport is set to true.	
Screenshot	N/A	
Properties	Property	Value
	Type	Variable: Integer
	Range	<= 18446744073709551615, >=0
	Default	0

3.2.3.17.2 AdcPowerStateReadyCbRef

Variable	AdcPowerStateReadyCbRef	
Description	Each instance of this parameter contains a reference to a power mode callback defined in a CDD or IoHwAbs component. This parameter shall only be configured if the parameter AdcLowPowerStatesSupport is set to true.	
Screenshot	N/A	
Properties	Property	Value
	Type	Variable: FunctionName
	Default	NULL_PTR

3.2.3.18 AdcEcucPartitionRef

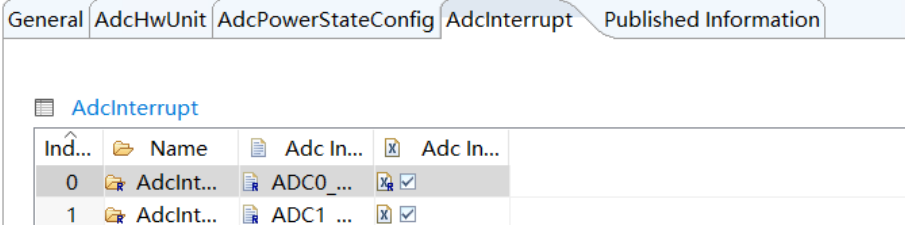
List	AdcEcucPartitionRef	
Description	Maps the Adc driver to zero or multiple ECUC partitions to make the modules API available in this partition. The Adc driver will operate as an independent instance in each of the partitions.	

Screenshot		
Properties	Property	List
	Type	Variable: REFERENCE
	Range	Reference to EcuC/EcucPartitionCollection/EcucPartition
	Min	0

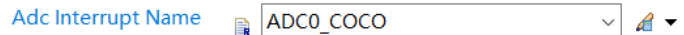
3.2.3.19 AdcKernelEcucPartitionRef

Variable	AdcEcucPartitionRef	
Description	Maps the ADC kernel to zero or one ECUC partition to assign the driver kernel to a certain core. The ECUC partition referenced is a subset of the ECUC partitions where the ADC driver is mapped to. Since the driver implementation only supports multi-core type II, this option is not supported.	
Screenshot		
Properties	Property	Value
	Type	Variable: REFERENCE
	Range	Reference to AdcEcucPartitionRef

3.2.3.20 AdcInterrupt

Container	AdcInterrupt	
Description	Selects whether the interrupt for each ADC Unit will be enabled. For each ADC HW unit, there are 2 interrupts that can be enabled: the End of Conversion and the Watchdog interrupts. These settings are used for optimizing the code size by removing the interrupt handling code for interrupts that are not needed.	
Screenshot		
Properties	Property	List
	Type	Map

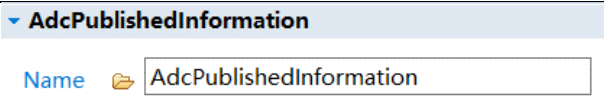
3.2.3.20.1 AdcInterruptSource

Variable	AdcInterruptSource	
Description	The name of the interrupt.	
Screenshot		
Properties	Property	Value
	Type	Variable: Enumeration
	Range	ADC0_COCO, ADC1_COCO
	Default	

3.2.3.20.2 [AdcInterruptEnable](#)

Variable	AdcInterruptEnable	
Description	Adds / removes the interrupt handling routine from the ADC driver code.	
Screenshot	Adc Interrupt Enable 	
Properties	Property	Value
	Type	Variable: Boolean
	Default	false

3.2.4 [AdcPublishedInformation](#)

Container	AdcPublishedInformation	
Description	Additional published parameters not covered by CommonPublishedInformation container. Note that these parameters do not have any configuration class setting, since they are published information.	
Screenshot		
Properties	N/A	

3.2.4.1 [AdcChannelValueSigned](#)

Variable	AdcChannelValueSigned	
Description	Information whether the result value of the ADC driver has sign information (true) or not (false). If the result shall be interpreted as signed value, it shall apply to C-language rules.	
Screenshot	Adc Channel Value Signed 	
Properties	Property	Value
	Type	Variable: Boolean_Label
	Value	false

3.2.4.2 [AdcGroupFirstChannelFixed](#)

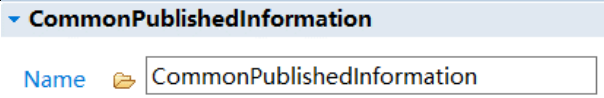
Variable	AdcGroupFirstChannelFixed	
Description	Information whether the first channel of an ADC Channel group can be configured (false) or is fixed (true) to a value determined by the ADC HW Unit.	
Screenshot	Adc Group First Channel Fixed 	
Properties	Property	Value
	Type	Variable: Boolean_Label
	Value	false

3.2.4.3 [AdcMaxChannelResolution](#)

Variable	AdcMaxChannelResolution	
Description	Maximum Channel resolution in bits (does not specify accuracy).	
Screenshot	Adc Max Channel Resolution 	
Properties	Property	Value
	Type	Variable: Integer
	Range	<=63, >=1

	Value	12
--	-------	----

3.2.5 CommonPublishedInformation

Container	CommonPublishedInformation
Description	Common container, aggregated by all modules. It contains published information about vendor and versions.
Screenshot	
Properties	N/A

3.2.5.1 ArReleaseMajorVersion

Variable	ArReleaseMajorVersion		
Description	Major version number of AUTOSAR specification on which the appropriate implementation is based on.		
Screenshot			
Properties	Property	Value	
	Type	Variable: Integer_Label	
	Value	4	

3.2.5.2 ArReleaseMinorVersion

Variable	ArReleaseMinorVersion	
Description	Minor version number of AUTOSAR specification on which the appropriate implementation is based on.	
Screenshot		
Properties	Property	Value
	Type	Variable: Integer_Label
	Value	6

3.2.5.3 ArReleaseRevisionVersion

Variable	ArReleaseRevisionVersion	
Description	Revision version number of AUTOSAR specification on which the appropriate implementation is based on.	
Screenshot	AUTOSAR Patch Version 0	
Properties	Property	Value
	Type	Variable: Integer_Label
	Value	0

3.2.5.4 ModuleId

Variable	ModuleId		
Description	Module ID of this module from Module List.		
Screenshot	Module ID 123		
Properties	Property	Value	
	Type	Variable: Integer_Label	
	Value	123	

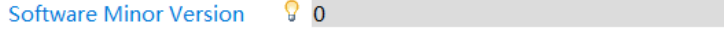
3.2.5.5 SwMajorVersion

Variable	SwMajorVersion	
Description	Major version number of the vendor specific implementation of the module. The numbering is vendor specific.	
Screenshot		
Properties	Property	Value
	Type	Variable: Integer_Label
	Value	0

3.2.5.6 SwMinorVersion

Variable	SwMinorVersion	
Description	Minor version number of the vendor specific implementation of the module. The numbering is vendor specific.	
Screenshot		
Properties	Property	Value
	Type	Variable: Integer_Label
	Value	4

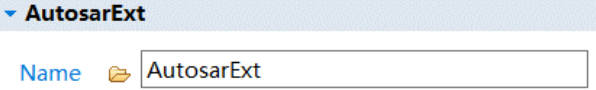
3.2.5.7 SwPatchVersion

Variable	SwPatchVersion	
Description	Patch level version number of the vendor specific implementation of the module. The numbering is vendor specific.	
Screenshot		
Properties	Property	Value
	Type	Variable: Integer_Label
	Value	0

3.2.5.8 VendorId

Variable	VendorId	
Description	Vendor ID of the dedicated implementation of this module according to the AUTOSAR vendor list.	
Screenshot		
Properties	Property	Value
	Type	Variable: Integer_Label
	Value	174

3.2.6 AutosarExt

Container	AutosarExt	
Description	Autosar Extension API settings.	
Screenshot		

Properties	N/A
-------------------	-----

3.2.6.1 AdcMulticoreSupport

Variable	AdcMulticoreSupport	
Description	This parameter globally enables the possibility to support multicore. If this parameter is enabled, at least one EcucPartition needs to be defined (in all variants).	
Screenshot	Adc Multicore Support  ☒  	
Properties	Property	Value
	Type	Variable: Boolean
	Default	False

3.2.6.2 AdcIsrPartitionMappingCheck

Variable	AdcIsrPartitionMappingCheck	
Description	The ISR function is checked according to the partition mapping to verify if the ISR is allowed to be routed to the partition.	
Screenshot	Adc Check Partition Mapping in ISR  ☐  	
Properties	Property	Value
	Type	Variable: Boolean
	Default	False

3.2.6.3 AdcDisableDemReportErrorStatus

Variable	AdcDisableDemReportErrorStatus	
Description	Enable/Disable Dem error reporting.	
Screenshot	Adc Disable Production Error Reporting  ☐  	
Properties	Property	Value
	Type	Variable: Boolean
	Default	False

3.2.6.4 AdcHardwareAverageOnce

Variable	AdcHardwareAverageOnce	
Description	Implementation Specific Parameter. Enable/Disable one time setting of the registers. If Enabled, the setting of the hardware average registers will be done only once in Adc_Init() function for the configured hardware unit.	
Screenshot	Adc Set Hardware Average Once  ☐  	
Properties	Property	Value
	Type	Variable: Boolean
	Default	False

3.2.6.5 AdcEnableDmaTrasferMode

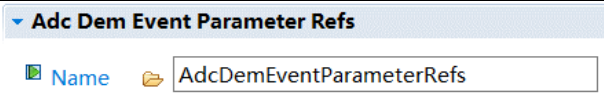
Variable	AdcEnableDmaTrasferMode	
Description	This parameter globally enables the possibility to configure DMA transfer for ADC converted data. If this	

	parameter is disabled then DMA handling code will be removed at pre-compile time and DMA transfer cannot be configured for any ADC unit in any variant. If this parameter is enabled then the DMA configuration code will not be removed. This is an Implementation Specific Parameter.	
Screenshot	Adc Global Enable DMA Transfer  	
Properties	Property	Value
	Type	Variable: Boolean
	Default	False

3.2.6.6 AdcEnableUserModeSupport

Variable	AdcEnableUserModeSupport	
Description	When this parameter is enabled, the ADC module will adapt to run from User Mode. Note: No special measures need to be taken to run ADC driver from user mode; The ADC driver code can be executed at any time from both supervisor and user mode.	
Screenshot	Enable Adc User Mode Support  	
Properties	Property	Value
	Type	Variable: Boolean
	Default	False

3.2.7 AdcDemEventParameterRefs

Container	AdcDemEventParameterRefs	
Description	Container for the references to DemEventParameter elements which shall be invoked using the API Serr_NotifyErrorEvent API in case the corresponding error occurs. The EventId is taken from the referenced DemEventParameter's DemEventId value. The standardized errors are provided in the container and can be extended by vendor specific error references.	
Screenshot		
Properties	N/A	

3.2.7.1 ADC_E_TIMEOUT

Variable	ADC_E_TIMEOUT	
Description	Major version number of AUTOSAR specification on which the appropriate implementation is based.	
Screenshot		
Properties	Property	Value
	Type	Variable: Reference
	Default	N/A

Chapter 4 Configuration Guides

4.1 Configuration Item Constraint

- 1) If the ADC priority mechanism is not ADC_PRIORITY_NONE, ADC queuing mechanism must be enabled.
- 2) You should select clock reference for the ADC hardware unit, and the clock frequency cannot exceed 30MHz.
- 3) If ADC group in back to back mode is not enabled, you must enable ADC group use channel delays.
- 4) You must add at least one channel in the AdcGroupDefinition. And if ADC group without interrupts is enabled, the channels added in AdcGroupDefinition cannot exceed 32.
- 5) If ADC group in back to back mode is enabled, and the channels added in AdcGroupDefinition are more than 8 channels, you shall configure the ADC delay next Ptimer to a non-zero value to ensure that the delay is more than the time needed for conversion of 8 channels.

4.2 ADC Usage Common Steps

- 1) Before configuring ADC, configure and enable ADC and Ptimer peripheral clocks in the MCU module.

McuPeripheralClockConfig_3	ADC1		FOSCDIV	1	0	1.2E7
McuPeripheralClockConfig_14	ADC0		FOSCDIV	1	0	1.2E7
McuPeripheralClockConfig_10	PTIMER0		OFF	1	0	0.0
McuPeripheralClockConfig_18	PTIMER1		OFF	1	0	0.0

- 2) Configure the PortPin mode for ADC channels.

PortPin_0	1	31	PTA31		ADC0_SE13_CMP0_IN3
PortPin_1	2	30	PTA30		ADC0_SE9
PortPin_2	3	29	PTA29		ADC0_SE8

- 3) Configure the general configurations for ADC, including the enabled ADC functions, the ADC queue and priority mechanism, and ADC queue depth.

▼ AdcGeneral

Name

AdcGeneral

Adc_DeInit API

Adc Enable Limit Check

Adc_StartStopGroup API

Adc Hw Trigger API

Adc_VersionInfo API

Adc Priority Mechanism

Adc Result Alignment

Adc Timeout: (1000 -> 4294967295)

Adc Dma Transferring Timeout: (1000 -> 4294967295)

Adc Max Queue Depth

☐ Adc Low Power States Support

ADC_PRIORITY_NONE

ADC_ALIGN_RIGHT

65535

65535

3

☐

Adc Development Error Detection

Adc Enable Queuing

Adc Group Notification Capability

Adc_ReadGroup API

Adc Power State Asynch Transition Mode

- 4) For the hardware units to be used, enable the interrupts for the units.

AdcInterrupt				
Ind...	Name	Adc Interr...	Adc Interrupt Ena...	
0	AdcInterrupt_0	ADC0_COCO	☒	
1	AdcInterrupt_1	ADC1_COCO	☒	

- 5) Add the hardware units to be used in AdcHwUnit configuration.

AdcHwUnit														
Ind...	Name	Adc H...	Adc L...	Adc Tr...	Adc Cl...	Adc V...	Adc Al...	Adc R...	ADC S...	Adc Pr...	Adc Tr...	Adc S...	Adc S...	Adc
0	AdcHwUnit_0	ADC0	0	ADC_IN...	/Mcu/...	INTERN...	RIGHT_...	BITS_12	60	PTIMER...	PTIMER...	8	16	Ac
1	AdcHwUnit_1	ADC1	1	ADC_IN...	/Mcu/...	INTERN...	RIGHT_...	BITS_12	60	PTIMER...	PTIMER...	8	16	Ac

- 6) For each ADC hardware unit, configure the general settings for the unit.

AdcHwUnit

Name

General | AdcChannel | AdcGroup

Adc Hardware Unit

Adc Logical Unit Id

Adc Source Clock

Adc Clock Reference

Adc Voltage Reference Selection

Adc Prescaler Value

Adc Dma Channel

Adc Alignment

Adc Resolution

ADC Startup Counter (2 -> 255)

Adc Pre-Trigger Select

Adc Trigger Select

AdcSampleTimeOptions

Name

Adc Sample Time Option 0 (2 -> 255)

Adc Sample Time Option 1 (2 -> 255)

Adc Sample Time Option 2 (2 -> 255)

Adc Sample Time Option 3 (2 -> 255)

AdcPtimerSettings

Name

Adc Ptimer Prescaler Divider Select

Adc Ptimer Multiplication Factor Select for Prescaler

Ptimer Sequence Error Interrupt Enable ☐

Ptimer Sequence Error Notification

- 7) Add the channels to be used in the hardware unit.

Ind...	Name	Adc L...	Adc Hardw...	Adc C...	Adc Channel Range ...	Adc C...	Adc C...	Adc C...	Adc Channel Sample Ti...	Adc C...
0	Resistor_0	1	ADC_CH4	☐	ADC_RANGE_ALWAYS	0	0	12	SAMPLE_TIME_OPTION_0	8
1	Resistor_1	0	ADC_CH0	☐	ADC_RANGE_ALWAYS	0	0	12	SAMPLE_TIME_OPTION_0	8
2	BLDC1_U_EMF	2	ADC_CH11	☐	ADC_RANGE_ALWAYS	0	0	12	SAMPLE_TIME_OPTION_0	8
3	BLDC1_V_EMF	3	ADC_CH12	☐	ADC_RANGE_ALWAYS	0	0	12	SAMPLE_TIME_OPTION_0	8
4	BLDC1_IMOTOR_U	4	ADC_CH8	☐	ADC_RANGE_ALWAYS	0	0	12	SAMPLE_TIME_OPTION_0	8
5	BLDC1_IMOTOR_V	5	ADC_CH9	☐	ADC_RANGE_ALWAYS	0	0	12	SAMPLE_TIME_OPTION_0	8
6	BLDC1_IMOTOR_S...	6	ADC_CH10	☐	ADC_RANGE_ALWAYS	0	0	12	SAMPLE_TIME_OPTION_0	8
7	BLDC1_BUS_VO	7	ADC_CH13	☐	ADC_RANGE_ALWAYS	0	0	12	SAMPLE_TIME_OPTION_0	8
8	BLDC2_W_EMF	8	ADC_CH28	☐	ADC_RANGE_ALWAYS	0	0	12	SAMPLE_TIME_OPTION_0	8
9	BLDC2_IMOTOR_W	9	ADC_CH16	☐	ADC_RANGE_ALWAYS	0	0	12	SAMPLE_TIME_OPTION_0	8
10	BLDC2_IMOTOR_S...	10	ADC_CH5	☐	ADC_RANGE_ALWAYS	0	0	12	SAMPLE_TIME_OPTION_0	8
11	BLDC2_BUS_VO	11	ADC_CH1	☐	ADC_RANGE_ALWAYS	0	0	12	SAMPLE_TIME_OPTION_0	8

8) For each channel, configure the channel parameters.

AdcChannel

Name

General

Adc Logical Channel ID

Adc Hardware Channel Id

Adc Channel Limit Check ☐

Adc Channel Range Select

Adc Channel High Limit

Adc Channel Low Limit

Adc Channel Conversion Time (0 -> 18446744073709551615)

Adc High Reference Voltage

Adc Low Reference Voltage

Adc Channel Resolution

Adc Channel Sample Time Option

Adc Channel Sample Time

9) In AutoSAR, ADC channels are accessed by groups, so the user should divide channels into groups.

Ind...	Name	Adc G...	Adc G...	Adc G...	Adc G...	Adc G...	Adc G...	Adc G...	Adc G...	Adc G...	Adc D...	Adc Pt...	Adc H...
0	AdcGroup_0	ADC_A...	ADC_C...	0	ADC_T...	ADC_S...	1	☐	☒	☒	0	0	☐
1	AdcGroup_1	ADC_A...	ADC_C...	1	ADC_T...	ADC_S...	1	☐	☒	☐	0	0	☐
2	AdcGroup_2	ADC_A...	ADC_C...	2	ADC_T...	ADC_S...	1	☐	☒	☐	0	0	☐

10) For each group, configure the group parameters.

AdcGroup

Name AdcGroup_0

General

AdcGroupDefinition

AdcChannelDelay

Adc Group Access Mode

ADC_ACCESS_MODE_STREAMING

Adc Group Conversion Mode

ADC_CONV_MODE_ONESHOT

Adc Group Id

0

Adc Group Priority

0

Adc Group Replacement

IC_GROUP_REPL_ABORT_RESTART

Adc Group Trigger Source

ADC_TRIGG_SRC_HW

Adc Group Trigger Signal

ADC_HW_TRIG_RISING_EDGE

Adc Group Trigger Timer (0 -> 18446744073709551615)

50

Adc Transfer Type

ADC_INTERRUPT

Adc Group Notification

IoHwAb_AdcNotification0

Adc Group Streaming Buffer Mode

ADC_STREAM_BUFFER_CIRCULAR

Adc Group Streaming Number Samples

3

Adc Group In Back to Back Mode

☒

Adc Group Uses Channel Delays

☒

Adc Delay Next Ptimer (0 -> 65535)

2000

Adc Ptimer Period For Continous Mode (0 -> 65535)

5000

AdcGroupHardwareAverageSettings

Name

AdcGroupHardwareAverageSettings

Adc Hardware Average Enable

☐

Adc Hardware Average Select

SAMPLES_4

11) Assign channels to the group.

AdcGroupDefinition

Ind...	AdcGroupDefinition
0	/Adc/Adc/AdcConfigSet/AdcHwUnit_0/Resistor_0
1	/Adc/Adc/AdcConfigSet/AdcHwUnit_0/Resistor_1

12) If channel delays are used for the group, the user shall also assign channel delays for the group.

AdcChannelDelay

Ind...	Adc Channel Delay
0	2000
1	4000